# fhe-intro-book

camcos-sjsu

2026-11-13

# Table of contents


Cover / Title Page 6

I Introduction 7

1 Setup 9

II General Functions 13
Some Basics . . . 14
Selection Vector (a.k.a. One-Hot, Masking, Binary Vector) . . . 14
Efficient Exponentiation (Repeated Squaring) . . . 15

2 Levels, Precision, and Time 16

3 Intravector Sums 18

4 Taylor Approximation (and Exponentiation) 21

5 The Square Root (and User Input) 23

6 Logarithm (First Glance) 24

III Division 25

7 Geometric Series 27

8 Newton's Method 28
8.0.1 Guessing a Starting Point . . . 28

9 A Hybrid Approach 32

10 Softmax Function 34

**IV Discrete Math** 35

**11 Sign Function** 37

**12 Logistic** 38

**13 Hyperbolic Tangent** 40

**14 Sign Applicability** 42
14.0.1 Inequality and Equality . . . . . 44

**V Statistics** 45

**15 Higher Moments** 47

**16 Difference Matrix Method** 49

**17 Mode** 51

**18 Median** 52

**19 Two Coefficients** 54

**20 Linear Regression** 56
20.0.1 Simple Linear Regression . . . . . 56
20.0.2 (Sketch of) Multiple Linear Regression . . . . . 57

**VI Sorting** 58
Comparison Based Sorting . . . . . 59
Rank and place . . . . . 59
Maximum and minimum . . . . . 59

**21 Bubble Sort** 60
21.0.1 PairSort . . . . . 60
21.0.2 The Bubble Sort Algorithm . . . . . 61

**22 Odd-Even Sort** 63

**23 Bitonic Sort** 65

**24 Rank Based Sort** 68
24.0.1 Methods . . . . . 68
24.0.2 Assumption . . . . . 68
24.0.3 Rank . . . . . 69

24.0.4 Rank-Based Sort . . . 70
24.0.5 Ties . . . 71

**25 Max Function** **73**

**VII Discrete Computational Models** **76**

**26 Roots of Unity** **78**

**27 Lookup Tables** **80**
27.0.1 Multivariable Interpolation . . . 80
27.0.2 Example: $n = 4$ . . . 81
27.0.3 Combining Inputs . . . 82
27.0.4 Exploiting Symmetry . . . 83
27.0.5 Exploiting Symmetry with Multivariate Interpolation . . . 85
27.0.6 Example: Construction and Contrasting of Modular Multiplication for n = 4 . . . 86

**28 A Binary Approach to Inequality and Sorting** **88**

**29 Comparison and Extended Binary Numbers** **89**

**30 Index To One-Hot** **92**

**31 Discrete sign and notes on sorting** **93**
31.0.1 Sign function . . . 93
31.0.2 Comparisons . . . 93
31.0.3 Bubble sort . . . 95
31.0.4 Sorting by rank . . . 96
31.0.5 Alternative idea for one-hot vectors . . . 96

**32 Summary** **97**

**VIIIOther Explorations** **98**

**33 Dot Products and Matrix Multiplication** **100**

**34 Shannon Entropy** **102**

**35 "Adaptive" Division** **104**

**36 Logarithm (Revisited)** **105**

**Appendices** **107**

**B References** **109**

# Cover / Title Page

**NOTE** From the Monday meeting: Perhaps we make this a homepage of sorts, with graphics and something from the team

# Part I

# Introduction

**CKKS** [1] (Jung Hee **C**heon, Andrey **K**im, Miran **K**im, and Yongsoo **S**ong) is a **fully homomorphic encryption (FHE)** scheme. FHE allows computations to be performed directly on encrypted data without first decrypting it, whereas traditional encryption requires decryption before computation. For example, a server can now extract insights from sensitive user data without having any knowledge of the data or the insights themselves.

(List was briefly edited to match with current state of book. To be rewritten with more flair.)

In this book, we explore functions that can be built under the CKKS homomorphic encryption scheme:

1. We introduce a simplified problem setup in Setup, then examine some initial functions which can be built from these base moves in General Functions.
2. In Division we explore division, which is surprisingly tricky but important.
3. In Discrete Math we study "discrete math" concepts that can be built from the sign function (which we do not have a good implementation for, but is an important primitive to have).
4. In Statistics we explore applications of the branching logic built in discrete math, which can support statistical functions like mode and median, and we look at statistical applications to linear regression.
5. In Sorting we explore sorting starting from a similar entry point as in conventional computing before quickly diverging down a different path based on constraints of the CKKS scheme.
6. In Discrete Computational we explore some approaches with a different (discrete) computational model, such as modular arithmetic and a more robust (but still potentially costly) Inequality function.
7. We end with some more speculative approaches to various topics in Other Explorations, including an iterative approach to log and some linear algebra.

CKKS has the advantage of natively supporting real- and complex-valued input, as opposed to other FHE schemes which are primarily designed to compute on exact integers. CKKS also makes use of "batching" techniques à la SIMD [2] (Single Instruction, Multiple Data) which increase efficiency through parallelized computations: many values are "packed" into a single ciphertext and computed upon simultaneously.

---

# 1 Setup

Our simplified CKKS setup is as follows:

1. Let $N$ be a power of 2 (concrete default: $N = 2^{16}$).
2. The users provide ciphertext $\mathsf{ct}$ with each such ciphertext encrypting $N/2$ complex numbers. In other words,
$$\mathsf{ct} = \mathtt{enc}(z_1, \dots, z_{N/2}).$$
3. As a default bound on the numbers involved, suppose the complex numbers have form $z_j = a_j + b_j i$ where $a_j, b_j \in \mathbb{R}$. We can assume the $a_j$ and $b_j$ are bounded above by $10^6$ and have precision $10^{-10}$.

As $\mathsf{ct}$ is just encrypting some data, we are allowed to make copies of $\mathsf{ct}$, save them, etc. **NOTE:** We are allowed to self-encrypt data which will be useful in further

Any further operations on $\mathsf{ct}$ are considered difficult by default, and we must instead "build" them from a limited set of basic "moves" given by the encryption scheme. These moves are as follows:

- **addition**: given ciphertext $\mathsf{ct}_1 = \mathtt{enc}(r_*)$, $\mathsf{ct}_2 = \mathtt{enc}(s_*)$, (where each $r_j, s_j \in \mathbb{C}$) we can generate a ciphertext
$$\mathsf{ct}' = \mathtt{enc}(r_1 + s_1, r_2 + s_2, \cdots, r_{N/2} + s_{N/2}).$$

Slot by slot, Figure 1.1 shows this.

- (element-wise) **multiplication**: given ciphertext $\mathsf{ct}_1 = \mathtt{enc}(r_*)$, $\mathsf{ct}_2 = \mathtt{enc}(s_*)$, we can generate a ciphertext
$$\mathsf{ct}' = \mathtt{enc}(r_1 s_1, r_2 s_2, \cdots, r_{N/2} s_{N/2}).$$

Figure 1.2 shows the same layout with products.

- **complex conjugation**: given ciphertext $\mathsf{ct} = \mathtt{enc}(z_*)$, we can generate a ciphertext
$$\mathsf{ct}' = \mathtt{enc}(\overline{z_1}, \dots, \overline{z_{N/2}}).$$
Recall that $\overline{a + bi} = a - bi$ for real numbers $a$ and $b$.

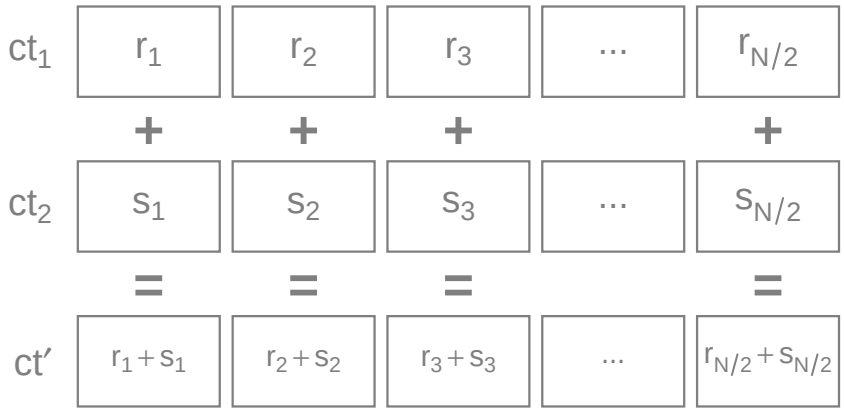



Figure 1.1: Slot-wise addition of two ciphertexts.

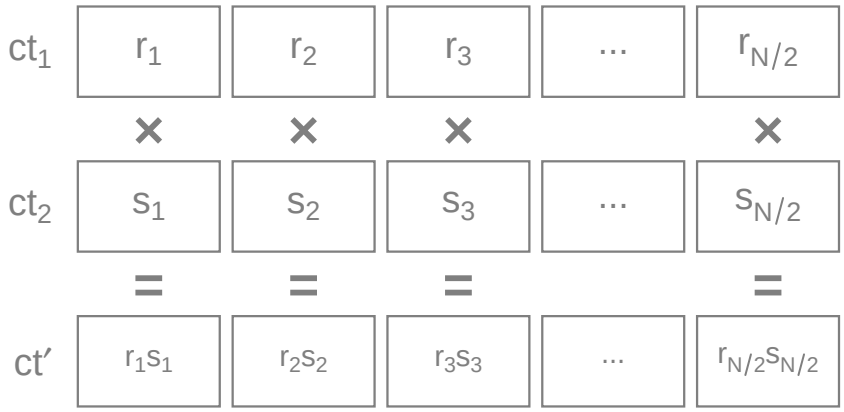



Figure 1.2: Slot-wise multiplication of two ciphertexts.

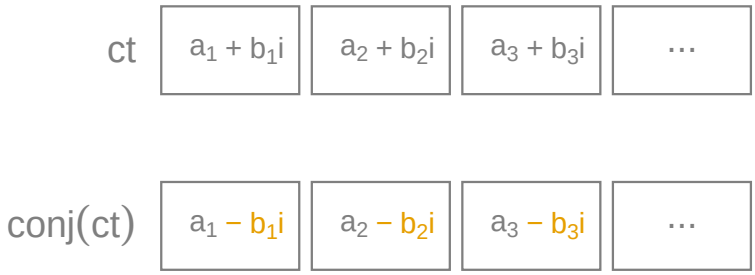



Figure 1.3: Complex conjugation of a ciphertext, slot by slot.

In Figure 1.3, each slot $z_j = a_j + b_j i$ keeps its real part and flips the sign of its imaginary part (orange).

- **cycle**: given a ciphertext $\mathtt{ct} = \mathtt{enc}(z_1, \ldots, z_{N/2})$, we can, for any $k \in \{1, 2, \ldots, N/2\}$, create a ciphertext
$$\mathtt{ct}' = \mathtt{enc}(z_k, \ldots, z_{N/2}, z_1, \ldots, z_{k-1}).$$

Figure 1.4 shows cycle with $k = 3$ on eight slots: the vector now starts at $z_3$, and the first two entries, $z_1$ and $z_2$ (orange), wrap around to the end.

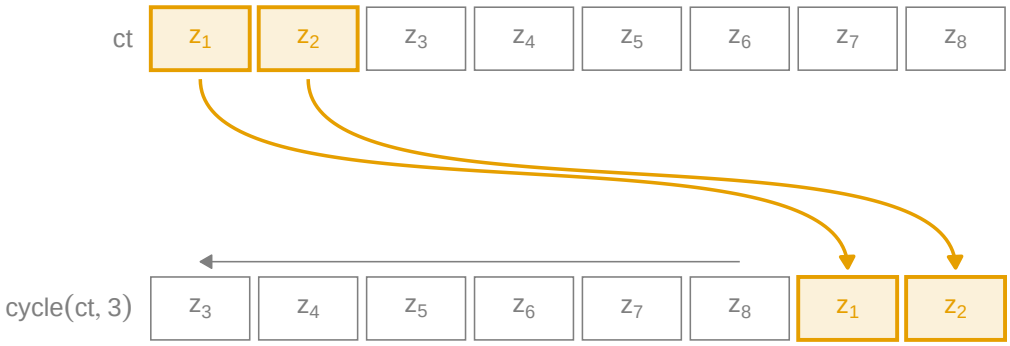



Figure 1.4: Cycle with $k = 3$ on eight slots.

This is not an exhaustive list of moves and part of open research is determining what other techniques are possible in CKKS. For the scope of our doc, we will assume this is the complete list of moves. We note two practical things that we will mostly ignore for the rest of our work:

1. We can assume these moves are fast but not negligible. Think of all of these moves as taking place on the order of milliseconds with addition being faster at microseconds.
2. In our scheme, numbers get worse in two ways as we compute on them: they accumulate noise (for instance floating point arithmetic accumulates error) and they also become potentially easier to decrypt by an adversary.

**Remarks**

1. For brevity, we will often write as shorthand $\texttt{enc}(z_*)$ or suppress the `enc` and discuss the vectors directly.

2. For many use cases (especially order-dependent notions like median, sign, etc.), we can assume that we are only dealing with real numbers (formally, the imaginary coordinate is 0). Since this is often clear from context, we omit disclaimers for clarity.

3. We are allowed to self-encrypt data, which is an important step in operating on our ciphertext directly.

# Part II

# General Functions

In this section, we design some general-use functions which may find place in subroutines used in the building of more complex functions: - sums - exponentiation - square root - log

We will also be using these functions as examples to show **techniques** that would be useful in other scenarios. Specifically:

1. Exponentiation shows the power of **Taylor series**.
2. The square root conveys the utility of **giving computation back to the user post-FHE**.

## Some Basics

We are allowed to generate encoded vectors of any constant, c.

$$ct_c = enc(c_1, c_2, ..., c_{N/2})$$

Note that $c_i$ can be distinct from $c_j$ This allows us to do computations with pre-computed constant vectors

$$c * enc(z_*) = enc(c, c, ..., c) * enc(z_*) = enc(cz_1, cz_2, ..., cz_{N/2})$$

Since we have multiplication, it is easy to divide by *known constants*, since we can multiply by their inverse. Specifically, we can take

$$\texttt{enc}(z_*) * \texttt{enc}(\frac{1}{c_*}) \rightarrow \texttt{enc}(\frac{z_1}{c_1}, ..., \frac{z_{N/2}}{c_{N/2}})$$

by precomputing constants $1/c_*$ from $c_*$.

### Selection Vector (a.k.a. One-Hot, Masking, Binary Vector)

Suppose we want certain elements in our ciphertext. We can encrypt a vector of ones (selected) and zeros (not selected) to decide on which elements are needed. For example, if we wanted to take only the first element of our vector $ct = enc(z_1, z_2, ... z_{N/2})$, all we need is one multiplication with our pre-encrypted $ct_0 = enc(1, 0, 0, ... 0)$ to obtain

$$ct_* = ct_0 * ct = enc(1, 0, 0, ..., 0) * enc(z_1, z_2, ..., z_{N/2}) = enc(z_1, 0, 0, ..., 0)$$

## Efficient Exponentiation (Repeated Squaring)

Suppose we wanted to obtain $ct_{x^k} = enc(x_1^k, \ldots, x_{N/2}^k)$.

While we can naively obtain $(x_1^k, \ldots, x_{N/2}^k)$ in $O(k)$ time by repeated multiplication, it is faster to compute via the "fast squaring trick":

Let X be a ciphertext vector of length N/2.

$X^2 = X * X$ (one operation)

$X^3 = X^2 * X$ (two operations)

$X^4 = X^2 * X^2$ (two operations)

$X^5 = X^2 * X^3$ (three operations)

$X^6 = X^3 * X^3$ (three operations) … and so on.

Therefore, we can obtain X to the $k^{th}$ exponent in $O(\log(k))$ operations.

# 2 Levels, Precision, and Time

As stated in the introduction, numbers get worse in three ways as we compute on them: noise, level, and time.

In this table,

- **precision** refers to a way an encryption of a number "decays with time" (as all known fully homomorphic schemes do). The basic idea is that after an operation, the new result will have some added mathematical noise.
  - For example, adding $Enc(r_*)$ and $Enc(s_*)$ to get $Enc(t_*)$, $t_i = r_i + s_i + e_i$ for some error $e_i$, so it is good to always assume our numbers are possibly off by some "noise" which accumulates as more operations are done on them.
  - This shouldn't be too surprising (e.g. in Python, 0.1 + 0.3 is not exactly 0.4).
  - As a default, we assume starting noise (lack of precision) is around $10^{-10}$.
- **(encryption) level** refers to the decryptability.
  - For example, assume a fresh ciphertext starts off at "level 35."
  - Multiplying two "level 30" numbers give "level 29" numbers, and level 0 numbers should be considered decryptable by a reasonable adversary and should be avoided.
- **time** is self-explanatory, but remember we are doing very parallelizable computation for the sake of applications.

| | + | x | conjugation / cycles |
|---|---|---|---|
| time | $50\mu s$ | $4ms$ (proportional to level) | $4ms$ |
| precision | noises add | noises stay constant | constant $10^{-10}$ worse |
| level | $a, b \to min(a, b)$ | $a, b \to \min(a, b) - 1$ | id |

Therefore, as we build additional functions, we must keep in mind the cost of each individual function, since each auxiliary function builds into more complex functions.

**Remark**

1. In the rest of the book, for the sake of simplicity, we will be optimizing for the amount of "operations", which is proportional to the costs we have in the table.
2. In Division and Discrete Math, you are thrown into the proverbial deep end as you will see costs skyrocket.

# 3 Intravector Sums

Suppose we wanted the sums of the *entries* of a vector; we call this the **intra-vector sum** to differentiate from the "inter-vector sum" of summing vectors, which is already supplied as a basic move. A naive approach would be to repeatedly use cycle to obtain encryptions of

$$(z_1, \dots, z_{N/2}), (z_2, \dots, z_{N/2}, z_1), \dots, (z_{N/2}, z_1, \dots, z_{N/2-1})$$

and then add them as in the inter-vector case. This puts the sum in every coordinate.

However, we can make this faster by noticing the following. Suppose we cycled by 1 and added instead, then we would be able to get:

$$\texttt{enc}(z_1 + z_2, z_2 + z_3, z_3 + z_4, \cdots)$$

then cycling by 2 and adding to itself would put $z_1 + z_2 + z_3 + z_4$ in the first position, but with only 2 additions instead of 3. Extending this method, we can sum all the numbers with $\log(N/2)$ operations.

Figure 3.1 shows the reduction tree for 8 slots: each row adds a copy of the row above cycled by 1, 2 and then 4, and because the cycles wrap around, every slot ends with the full sum, not just the first.

The savings grow quickly with the number of slots (Figure 3.2). For $N = 2^{16}$ the full vector has $N/2 = 2^{15}$ slots, and the sum takes 15 cycles instead of 32,767.

Here are a couple of general consequences of being able to sum:

1. **dot product**: given encrypted vectors of $c_*$ and $d_*$, we can multiply to get $\texttt{enc}(c_1 d_1, c_2 d_2, ...)$, and then sum to get the dot product.
2. **(complex) inner product**: since we can take the conjugate of $d_*$, we can compute the complex inner product
$$c_1 \overline{d_1} + \cdots + c_{N/2} \overline{d_{N/2}}.$$

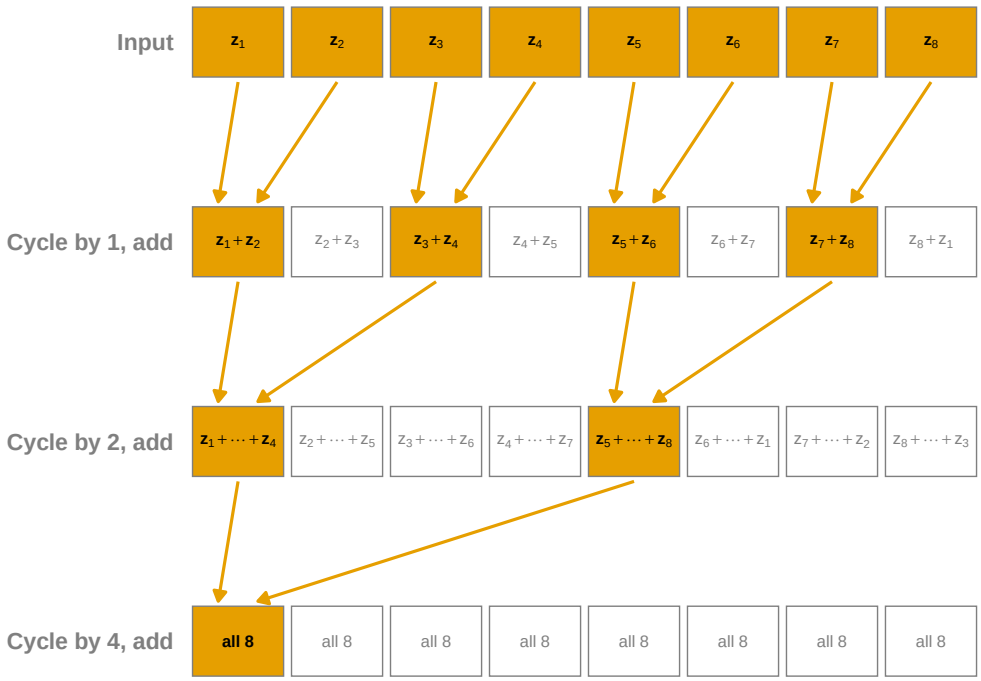



Figure 3.1: Summing eight slots with three cycle-and-adds, shifting by 1, 2, then 4, forms a binary tree.

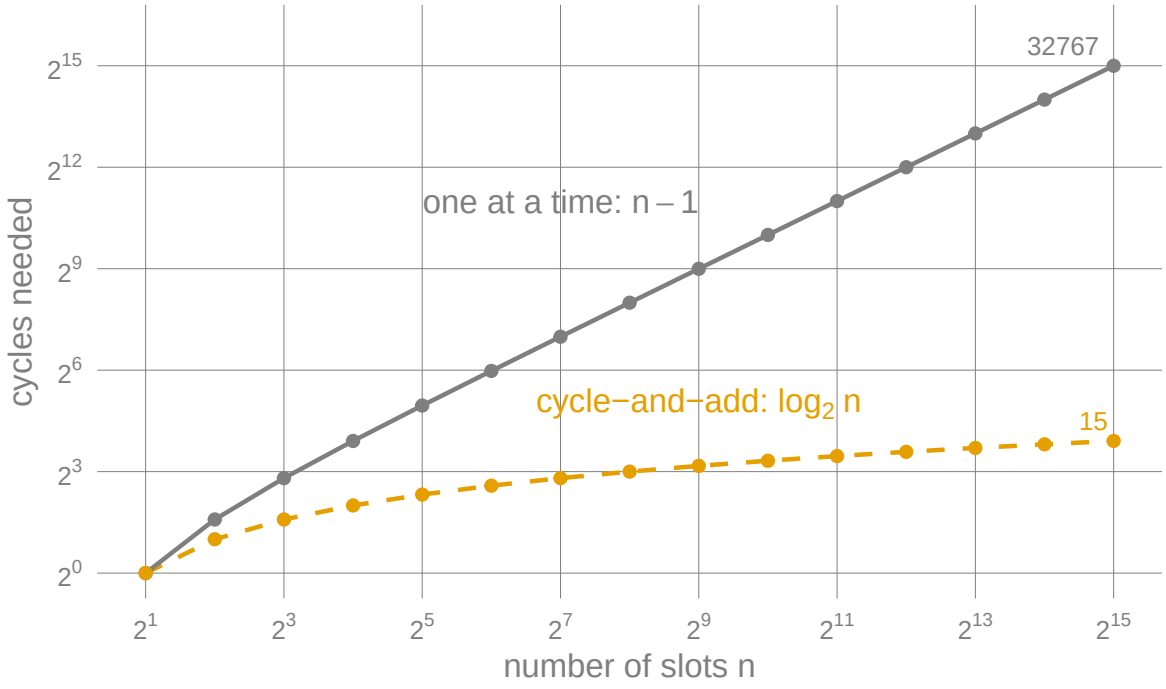



Figure 3.2: Cycles needed to sum $n$ slots one at a time versus by cycle-and-add.

# 4 Taylor Approximation (and Exponentiation)

Beyond our base moves, many functions which are of interest to us can be approximated with **Taylor approximation**: approximate a function by the first few terms of its power series. This is fairly straightforward in CKKS, involving only a combination of basic moves: additions, powers (multiplications), and scalar divisions.

We show the Taylor approximation of the **exponential function** as an illustrative example:

$$e^z \approx \sum_{k=0}^{N} \frac{z^k}{k!} = 1 + z + \frac{z^2}{2!} + \frac{z^3}{3!} + \cdots + \frac{z^N}{N!}$$

Figure 4.1 compares these approximations with $e^z$. Each one is accurate near $z = 0$ and drifts away from it, and a larger $N$ stays accurate over a wider range.

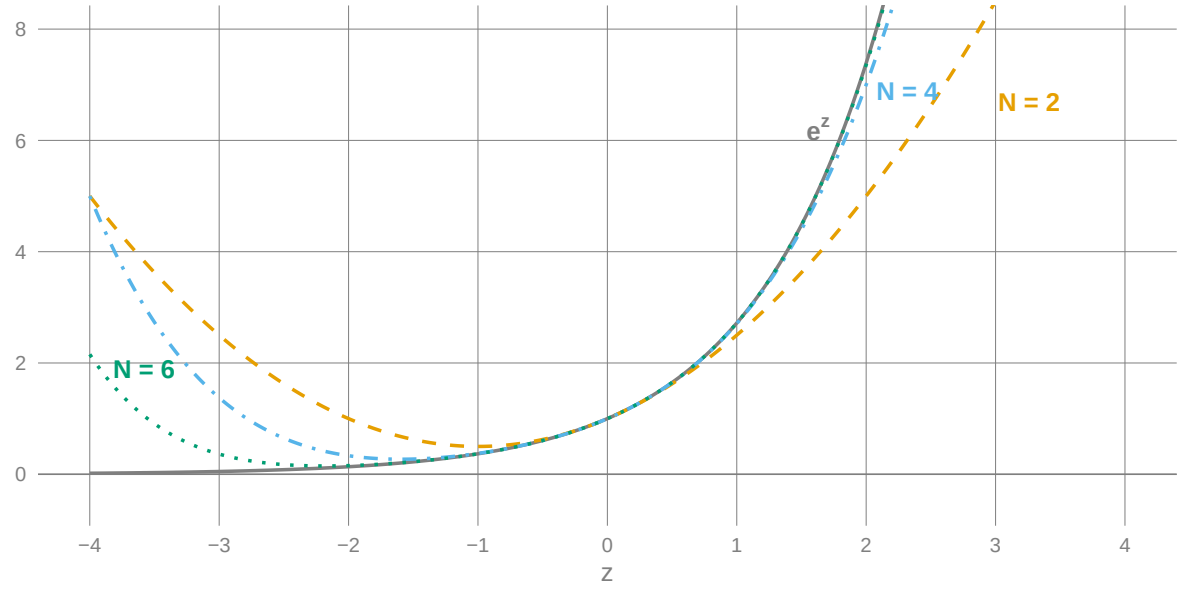



Figure 4.1: Taylor approximations of $e^z$ for $N = 2, 4, 6$ compared with $e^z$.

Our main thoughts on Taylor approximation are:

1. In a purely mathematical setting, a higher $N$ results in higher accuracy, yet in FHE, too large of an $N$ can result in lower accuracy as noise accumulates. We desire a middle-ground $N$ that produces a close approximation while mitigating cost and noise concerns.

2. We need to have a good sense of where our input lies for convergence assumptions. In general, there is a radius of convergence for $z$ (or $x$), which in the exponential case is infinite. But even within the radius of convergence you may in practice want the assumed domain to be even tighter for faster convergence.

3. In several cases, it seems possible to employ "range reduction" techniques to tighten domains. For example, here we can make $x$ smaller using $e^x = (e^{x/k})^k$ – in other words, we approximate $e^{x/k}$ then compute the $k$-th power of the result. Even better, when applicable, we may just pass the $k$-th power computation to the client.

# 5 The Square Root (and User Input)

We can similarly take the square root by expanding a Taylor series. The square root has no Taylor series at $x = 0$, but can be expanded elsewhere, such as at $x = 1$:

$$\sqrt{1-x} = 1 - \frac{x}{2} - \frac{x^2}{8} - \frac{x^3}{16} - \frac{5x^4}{128} - \frac{7x^5}{256} + O(x^6) + \cdots$$

which converges when $|x| \leq 1$.

However, if we give the user $x$, it might be easier for the user to compute $\sqrt{x}$ on the client side! Thus, frequently we might just consider *not* doing some computation over FHE, given business logic allows us to do so.

To summarize,

1. We can (and at times must) take Taylor series around points other than $x = 0$,
2. However we frequently may be able to skirt the problem using user output (and input). This is an important idea going forward to reduce computational cost, but will be tabled for now. Note that this is only feasible when giving the input back to the user does not compromise security.

**Remark**

We also explored e.g. the Babylonian approximation method[^bab] for taking the square root, but that would involve dividing by $x$, which is expensive. See Division for more information.

# 6 Logarithm (First Glance)

As usual, the Taylor approximation is a good default, and immediately doable with basic moves here: additions, powers (multiplications), and scalar divisions.

Importantly, as $\log_2(x)$ is undefined at 0, there is no Taylor expansion centered at 0 to use for approximation. We bypass this by approximating around $x = 1$, writing $x = 1 + u$. Now we can use a standard Taylor series expansion:

$$\log_2(1+u) = \log_2(x) \approx \frac{1}{\log_e(2)} \sum_{k=1}^{N} (-1)^{k+1} \frac{(x-1)^k}{k}$$

$$= \frac{1}{\log_e(2)} \left[ (x-1) - \frac{(x-1)^2}{2} + \frac{(x-1)^3}{3} - \frac{(x-1)^4}{4} + ... \right].$$

The series only converges for $x \in (0, 2]$.

**Remark**

In Discrete Math we will explore another approach to log which is not ripe now for practical use, but can give ideas on how to push FHE further.

# Part III

# Division

In this section, we tackle **division**, which is surprisingly tricky in our specialized context. Here, by division we mean dividing by an encrypted unknown value, such as wanting $\texttt{enc}(\frac{1}{z_*})$ given $\texttt{enc}(z_*)$. This is not obvious and cannot be precomputed.

**Remark**

Of course, this depends on context. For example, in a hypothetical scenario where we only ever divide by *user input*, we can just ask the users to supply inverses to their own numbers (that is, if every time we get an encryption of $x_i$ we also get $\texttt{enc}(1/x_i)$), then this is not a hard problem.

If we unlock division, we are then able to compute many other useful functions. More than just being able to obtain the function, however, our main contribution here is comparing different types of approaches to division, many of which may be useful for future endeavors (specifically, **Newton's Method**, some variants, and our more speculative adaptive Newton's Method.

# 7 Geometric Series

As usual, our initial instinct should be Taylor series: supposing $z_i$ is not far from 1, we can expand

$$\frac{1}{z} = \frac{1}{(1-(1-z))} = 1 + (1-z) + (1-z)^2 + \cdots$$

Explicitly, this means we can first compute

$$\texttt{enc}(z'_*) := \texttt{enc}(1 - z_*)$$

by subtracting the vector from a precomputed constant vector $\texttt{enc}(1, \ldots, 1)$, then compute powers and add

$$\texttt{enc}(1, \cdots, 1) + \texttt{enc}(z'_*) + \texttt{enc}((z'_*)^2) + \texttt{enc}((z'_*)^3) + \cdots$$

The partial sum of the geometric series method has some potential issues. For one, if the value being inverted is too far ($> 1$) away from 1, then the series will not converge. We can employ some algebraic tricks to shift the convergence interval. Another issue, assuming convergence, is the cost associated with repeated multiplications within the exponential terms.

For instance, suppose we require an error of no greater than $10^{-10}$ for one of these approximations, the number of partial sum terms needed for this much accuracy is shown:

**Sample Series Convergences**

| $z_i$ | $1/z_i$ (Target) | $r = 1 - z_i$ | Iterations to $\epsilon \leq 10^{-10}$ |
|---|---|---|---|
| 0.1482 | 6.746761986503 | 0.8518 | 155 |
| 0.5656 | 1.767918114060 | 0.4344 | 28 |
| 0.5941 | 1.683233435953 | 0.4059 | 26 |
| 0.6340 | 1.577252912141 | 0.3660 | 23 |
| 1.3748 | 0.727355832051 | −0.3748 | 23 |

Note the correlation of $1/z_i$ with the number of iterations required: the count grows with $|r|$, since the error after $n$ terms shrinks like $|r|^n$, and it blows up as $|r|$ approaches 1.

NULL

# 8 Newton's Method

We considered another approach with Newton's method, which finds the roots of a function $f(x)$. In this case, we can find the decimal value of $x^* = \frac{1}{d}$ so that $f(x) = \frac{1}{x} - x^* = 0$. Think of $x^*$ as an unknown decimal that we're looking for.

We use Newton's formula $f(x^*) \approx f(x_n) + f'(x_n)(x^* - x_n)$, and set $f(x^*) = 0$ :

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

$$x_n - \frac{f(x_n)}{f'(x_n)} = x_n - \frac{\frac{1}{x_n} - d}{\frac{-1}{x_n^2}} = x_n + x_n^2(\frac{1}{x_n} - d) = x_n + (x_n - x_n^2 d)$$

$$x_{n+1} = 2x_n - x_n^2 d$$

**Remark**

See an independently developed discussion developed by [3] Prant et al.

This allows us to approximate $1/x$ in a chain of $2n$ multiplications and $n$ additions. We can do a simple test of approximating $\frac{1}{48}$ with Newton's method versus a geometric series as follows:

Figure 8.1 measures accuracy by the number of correct decimal digits, defined as $-\log_{10}|\text{error}|$: an error of $10^{-4}$ is 4 correct digits. Both methods start from $x_0 = 1/64$ and are plotted after each iteration.

The one catch with Newton's method is that in order to be effective, it has to initially guess reasonably close to the root.

### 8.0.1 Guessing a Starting Point

Naively, one (impossible) approach would be to make some different buckets of $n$ based on powers of 2:

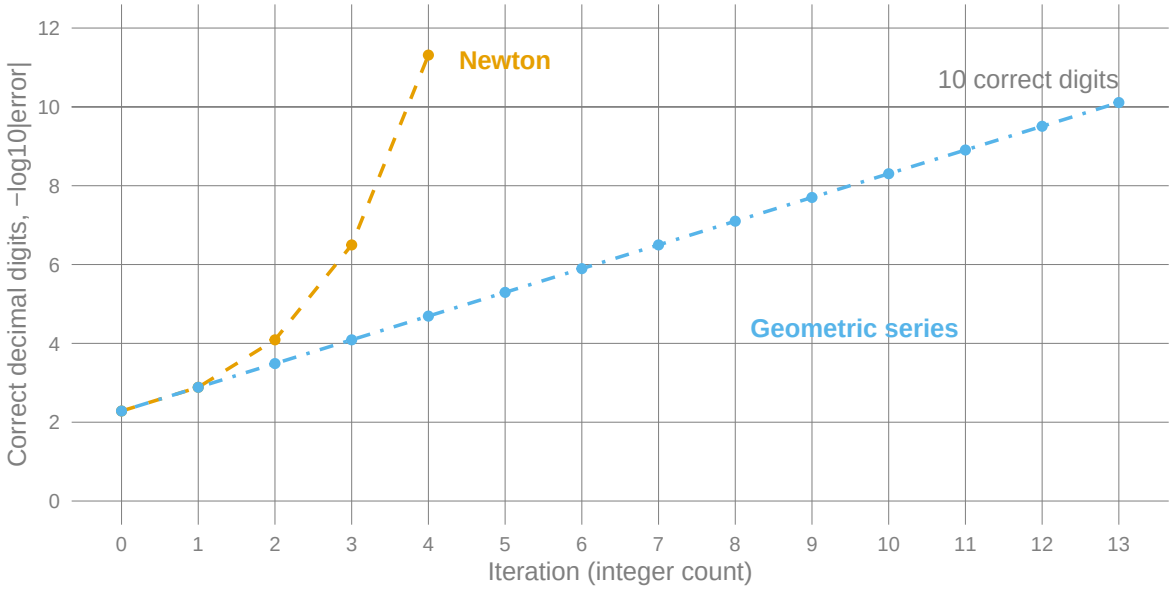



Figure 8.1: Correct decimal digits, $-\log_{10}|x_n - 1/48|$, after each iteration for Newton's method and a scaled geometric series approximating 1/48, both started from $x_0 = 1/64$ (plaintext).

| n | $ct_{2^n}$ | $ct_{2^{-n}}$ |
|---|---|---|
| 0 | 1 | 1 |
| 1 | 2 | $\frac{1}{2}$ |
| 2 | 4 | $\frac{1}{4}$ |
| ⋮ | ⋮ | ⋮ |
| 33 | $2^{33}$ | $2^{-33}$ |

Note: we only go to $n = 33$ since $\log_2(10^{10}) \approx 33$, where $10^{-10}$ is the approximate "round to zero" threshold. This would need a tweak to include negatives.

What we want to do is to figure out where the divisor belongs based on where $n$ belongs, and then make our guess $x_0$ be based on the corresponding boundary bucket. Sadly, this does not work because we cannot (yet) do different executions depending on branching logic.

Unless we try some more complex methods, we will need to come up with a universal $x_0$ guess [4] that will work regardless of what $\frac{1}{x}$ is.

Fortunately, the basin of convergence [5] for Newton's method approximation of $\frac{1}{x}$ is $(0, \frac{2}{x})$ if $x > 0$ and $(\frac{2}{x}, 0)$ if $x < 0$.

1. To guarantee convergence we can simply pick the closest number to zero that does not get eliminated by rounding, maybe $\approx 10^{-10}$ or $-(10^{-10})$, depending on the sign of $\frac{1}{x}$ (which needs to be known), every time.

2. If we want to account for the sign and have a Sign function, we would be able to capture it with something like $(x < 0) * (x_0 = -(10^{-10})) + (x > 0) * (x_0 = 10^{-10})$. See Section IV for similar methods.

Figure 8.2 shows the iteration $g(x) = 2x - dx^2$ for $d = 11$ as a staircase between the curve and the line $y = x$.

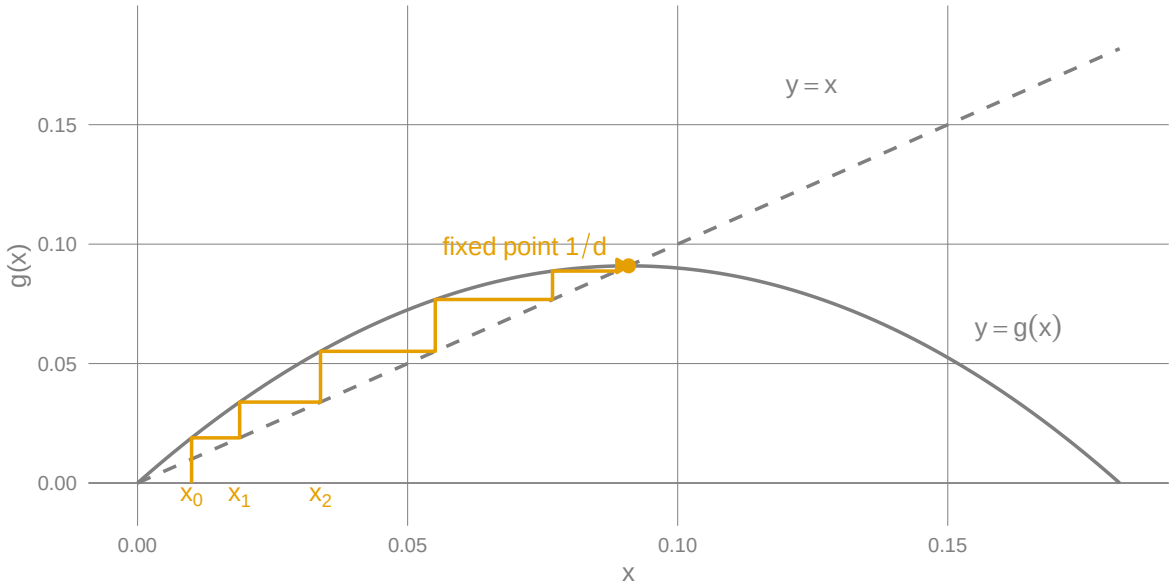



Figure 8.2: Cobweb diagram of the Newton iteration $x_{n+1} = g(x_n) = 2x_n - dx_n^2$ for $d = 11$ (plaintext), started at $x_0 = 0.01$. The iterates climb to the fixed point $1/d = 1/11$, where the parabola peaks and crosses $y = x$.

NULL

3. If we have additional knowledge about the actual range of values, e.g. if we know that values are in $[a, b]$ then we can uuse something like the geometric mean $1/\sqrt{ab}$. (concretely, if weight is always between 50 and 5000 pounds, we can take an initial guess of $1/500$).

**Disclaimers**

1. The "closest number to 0" method might cause overflow issues. This would require testing.

2. Starting so close to zero makes a very steep derivative of $\frac{1}{x}$ at that point, leading to the second guess $x_1$ being within the error bound of $x_0$: a sort of false convergence. Even attempting a remedy with stricter difference criteria leads to 30+ iterations before true convergence, wiping out some of the initial savings, which in fact may be more expensive than the geometric series.

# 9 A Hybrid Approach

The sum of the geometric series does most of its heavy lifting in the first few iterations, and then closes in slowly and expensively. Newton's method has the opposite problem: the only universal good starting point would be too far to the edge of the basin, and would waste the bulk of its moves in the beginning, making almost no progress toward sufficient convergence.

Perhaps the two approaches could be combined into a hybrid. The first one or two terms of the sum of the geometric series are less expensive and put us strongly within the basin of convergence, at which point we can put Newton in the driver's seat. Our opening guess for Newton's $x_0$ would be guaranteed to be in the convergence basin and well ahead of the default universal guess.

NULL

Figure 9.1 compares the three approaches for $d = 11$. The horizontal axis counts the ciphertext multiplications each approach has spent, which is the expensive operation in CKKS, and the vertical axis shows how many decimal digits of error remain (each step down is a factor of 10 smaller).

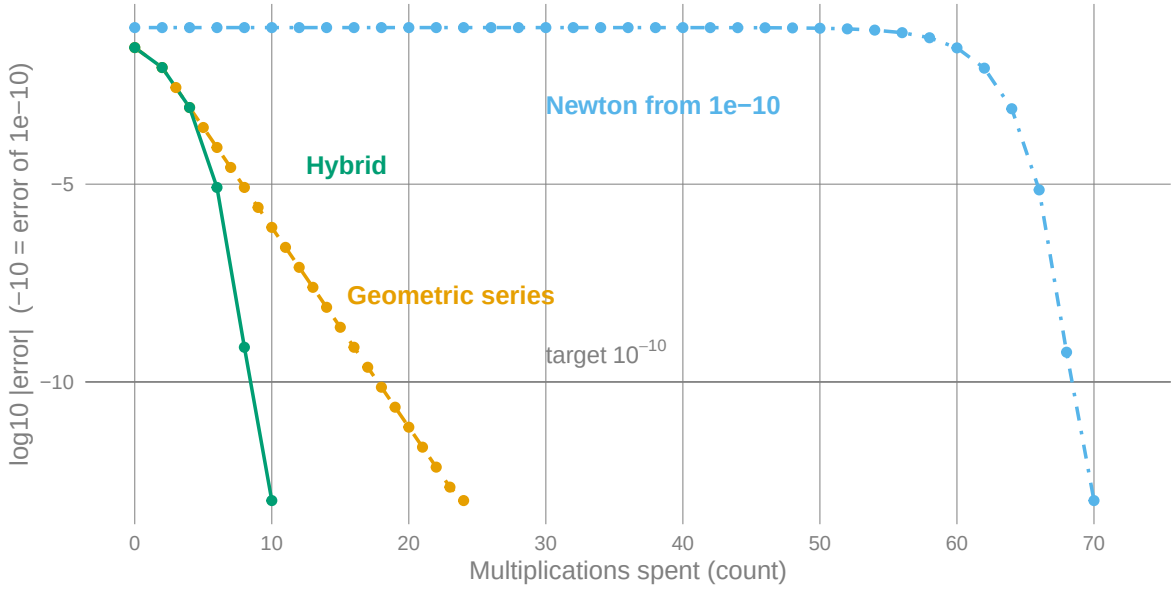



Figure 9.1: Plaintext error of 1/11 against the number of multiplications spent for the geometric series, Newton's method from the universal start $10^{-10}$, and the hybrid that starts Newton from the first geometric term 1/16. Cost basis: Newton costs 2 multiplications per step ($dx_n$, then $x_n(2 - dx_n)$); the partial sum through $r^n$ with $r = 5/16$ costs $n + 1$ (one for $r$, $n - 1$ for the powers, one to scale by 1/16); the first term 1/16 is a constant and costs 0. Errors below $10^{-13}$ are drawn at $10^{-13}$.

# 10 Softmax Function

A good example of a function that is directly unlocked by division is the **softmax function**, which takes a vector $x = (x_1, \dots, l_k) \in \mathbb{R}_+^{\Bbbk}$ and returns a probability distribution $r(x) = (p_1, \dots, p_k) \in (0, 1)^k$ as follows:

$$\rho(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

The function is important for many applications, but especially recently in neural nets for turning vectors into probability distributions for decision making.

The basic strategy is:

1. Make a vector $x'$ where $x'_i = e^{x_i}$ (this can be done by Taylor approximation or by asking the user to supply the exponentials);
2. Make a vector $s$ where all $s_i = \texttt{sum}(x_i)$;
3. Lastly, divide $x'$ by $s$.

**Concrete Example**

Take $z = (4, 2, 24, 6)$. We first exponentiate the vector's entries, or ideally pass to client: $(e^4, e^2, e^{24}, e^6)$. Separately compute the sum $S = e^4 + e^2 + e^{24} + e^6$. We now normalize our vector to get $(e^4/S, e^2/S, e^{24}/S, e^6/S)$ which rounds off to approximately (0, 0, 1, 0).

# Part IV

# Discrete Math

In this section we study some "discrete" operations involving comparisons and sorting. At the basis of everything, we need **sign**, **equality**, and **inequality** functions. The sign function seems the most fundamental since it can be used to produce the other two naturally, though it's essentially the same as inequality.

**Disclaimer**

Taylor approximations for sign run into many (unavoidable) issues, and can only be used if we have a very tight understanding of the domain of the function. We stress that what we do in the remainder of this section (and other parts of our work that rely on sign functions) is speculative because everything rests on an idealized sign function, which we do not quite have in practice. Conservatively, we should interpret our following work as an exploration of what we can unlock if we have an idealized sign function.

With this disclaimer, we perform the following explorations:

1. We try to use series approximations for sign anyway, and quantify how close the approximations are (we suspect it would be usable for very small-scale work, but not beyond that).
2. Assuming an "idealized sign", we use PairSort (a "local swap" / "bubble") as a building block for **sorting** (and as a bonus, computing **max**/min type of summary statistics). The functions we will implement are represented in the following "tech tree".
3. We provide a speculative approach to sign / inequality type of functions with binary arithmetic later on.

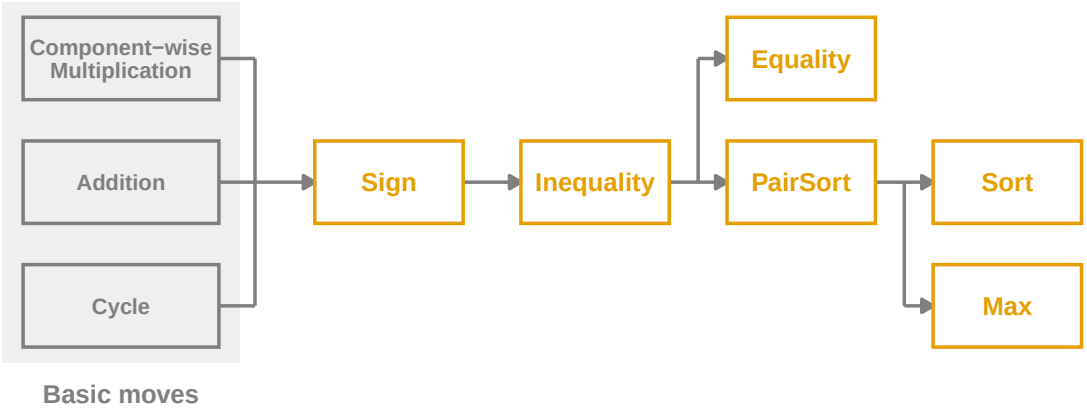



Figure 10.1: The tech tree for this section. The basic moves in the grey band build up to the functions we implement, shown in bold orange.

# 11 Sign Function

In many function constructions it would be of great value to have conditional logic, however, this is not immediate or straightforward in the CKKS setup. Though, we see an opportunity to do things in a "branchless" way via algebra. As a proof-of-concept (and a very important tool), we show computational approaches for a **sign** function that gives one of two possible values. Later we will use this function to build more powerful functions.

1. We think of a "basic" sign function as a function $\texttt{sign}(x)$ assigning a value of 0 or 1 depending on whether the input is respectively negative or positive.
   - It is useful to think of the output value 1/2 as a "necessary" evil for when the input is extremely close to 0 (this will basically be a necessity since our computations are inherently continuous). In practice, we will try to avoid those values as purposefully appearing in our computations.
2. We can generalize into any Heaviside-type function[1] that takes two possible values via the following:
   $$\texttt{sign}_{a,b,c}(x) = a + (b-a)\texttt{sign}(x-c),$$
   which gives values $a$ for $x < c$, $b$ for $x > c$, and $(a+b)/2$ for $x = c$ (though it is advised to avoid being dependent on the "true equality" case). We can then think of the "original" $\texttt{sign}(\texttt{x})$ as $\texttt{sign}_{0,1,0}(\texttt{x})$.

To approximate this naturally discontinuous function, we present two smooth avenues in the sigmoid family:

[1] **Heaviside step function** available from https://en.wikipedia.org/wiki/Heaviside_step_function

# 12 Logistic

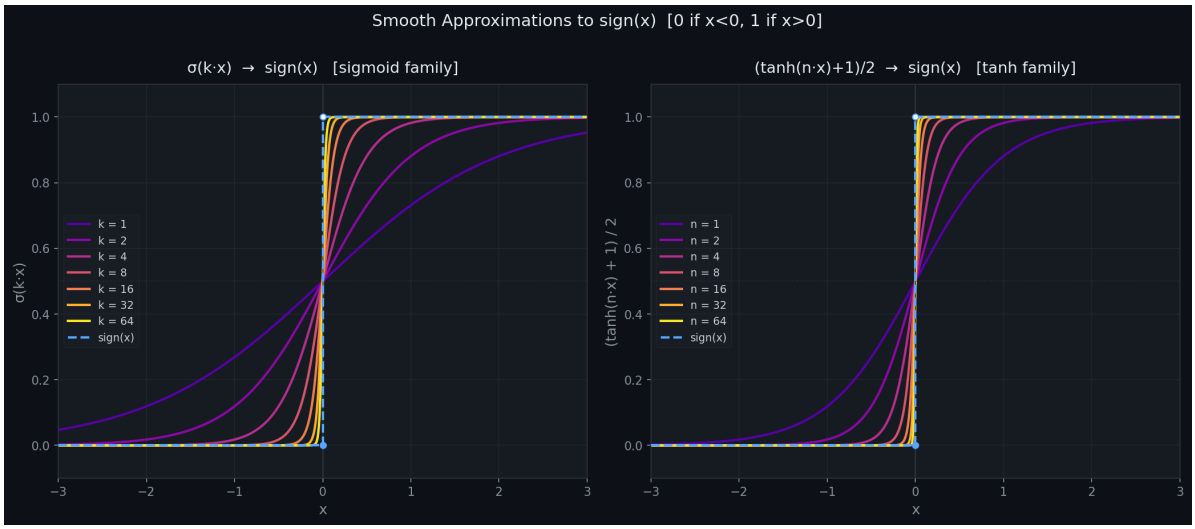



Figure 12.1: sign_approx

A **sigmoid function**[1] is an S-shaped curve that smoothly transitions between two values, essentially converging to the discontinuous sign function as its steepness increases, making such functions natural choices for our approximation. A common instance of a sigmoid function is the **logistic function**, which has standard definition $\sigma(x) = \frac{1}{1+e^{-x}}$. We begin our approximation of logistic using the series expansion for exponentiation:

$$e^x \approx 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + ...$$

$$1 + e^{-x} \approx 2 - x + \frac{x^2}{2!} - \frac{x^3}{3!} + \frac{x^4}{4!} - \frac{x^5}{5!} + \frac{x^6}{6!} - \frac{x^7}{7!} + \frac{x^8}{8!} - \frac{x^9}{9!} + ...$$

$$\sigma(x) = \frac{1}{1+e^{-x}} \approx \frac{1}{2 - x + \frac{x^2}{2!} - \frac{x^3}{3!} + \frac{x^4}{4!} - \frac{x^5}{5!} + \frac{x^6}{6!} - \frac{x^7}{7!} + \frac{x^8}{8!} - \frac{x^9}{9!}}$$

[1] **Sigmoid function** available from https://en.wikipedia.org/wiki/Sigmoid_function

$$\sigma(x) \approx \frac{1}{2} \cdot \frac{1}{1 + \left(-\frac{x}{2} + \frac{x^2}{4} - \frac{x^3}{12} + \frac{x^4}{48} - \frac{x^5}{240} + \frac{x^6}{1440} - \frac{x^7}{10080} + \frac{x^8}{80640} - \frac{x^9}{725760}\right)}$$

Denoting the entire term inside the parentheses as $r$, i.e. $r = -\frac{x}{2} + \frac{x^2}{4} - \frac{x^3}{12} + \ldots$ , we can then use the geometric series identity $\frac{1}{1+r} = 1 - r + r^2 - r^3 + \ldots$ ; computing powers of $r$ then canceling/ simplifying, we arrive at

$$\sigma(x) \approx \frac{725760 + 362880 * x - 30240 * x^3 + 3024 * x^5 - 306 * x^7 + 31 * x^9}{1451520} - \ldots$$

which reduces to an ordinary polynomial multiplied with an integer reciprocal, legal under the CKKS moves! However… 1. in theory, this series will only converge within $0 < x < \pi$. This is for two separate reasons, one being that the "$r$" term of the geometric series is $e^{-x}$, which is greater than 1 for $x < 0$ and will diverge. The other reason is that the nearest singularity to 0 is at $\pi$. 2. in practice, the approximation error is very large, and the function will quickly diverge.

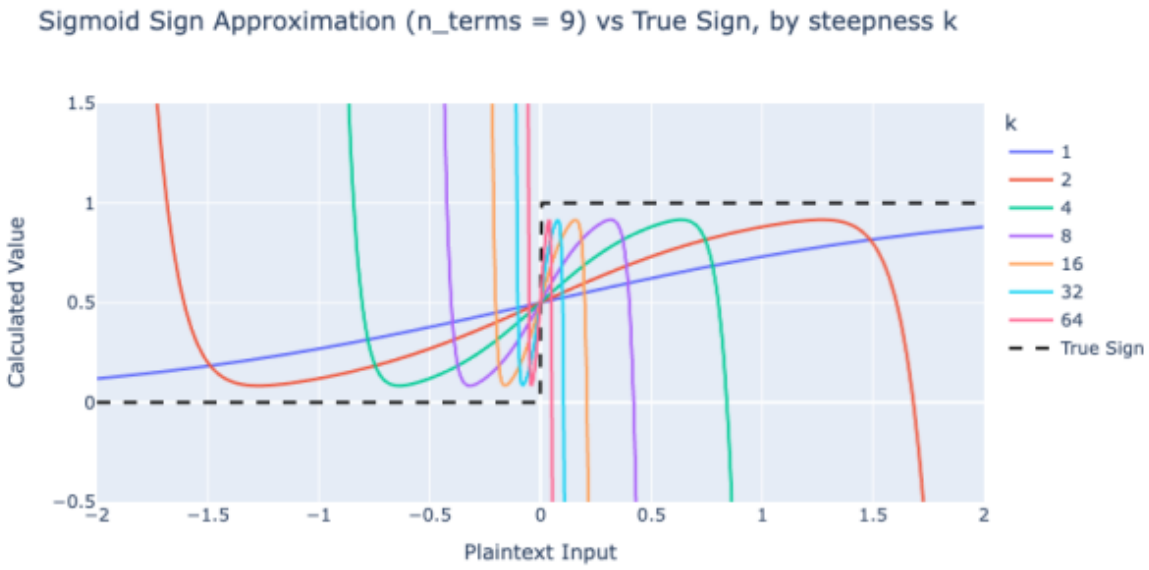



**Figure**: different graphs of the function $\sigma(kx)$ with steepness parameter $k$.

# 13 Hyperbolic Tangent

Also belonging to the sigmoid family is the hyperbolic tangent function[1]:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

where $e^{-2x} \approx 1 - 2x + \frac{4x^2}{2!} - \frac{8x^3}{3!} + \cdots$ ,

$$\tanh(x) \approx \frac{2x - \frac{4x^2}{2!} + \frac{8x^3}{3!} - \cdots}{2 - 2x + \frac{4x^2}{2!} - \frac{8x^3}{3!} + \cdots} = \frac{2x}{2} * \frac{1 - \frac{2x}{2!} + \frac{4x^2}{3!} - \cdots}{1 - x + \frac{2x^2}{2!} - \frac{4x^3}{3!} + \cdots}$$

Using the same trick as we did with the logistic expansion, we build a geometric series:

$$r = \Big( - x + \frac{2x^2}{2!} - \frac{4x^3}{3!} + \cdots \Big)$$

$$\frac{1}{1 + r} = \frac{1}{1 - (-r)} = 1 - r + r^2 - r^3 + \cdots$$

After plugging back in,

$$\tanh(x) \approx x * \Big(1 - \frac{2x}{2!} + \frac{4x^2}{3!} - \cdots \Big) * (1 - r + r^2 - r^3 + \cdots)$$

Computing powers of $r$, simplifying terms then finding the least common multiple for up to power 9,

$$\tanh(x) \approx \frac{2835x - 945x^3 + 378x^5 - 153x^7 + 62x^9}{2835} - \cdots$$

This series has a radius of convergence $\pi/2$ (the nearest singularities of tanh are at $x = \pm i\pi/2$, where $e^{-2x} = -1$), so it converges only for $-\pi/2 < x < \pi/2$, half the interval of the logistic series. As we show below, this is only a rescaling, not a real disadvantage.

[1] **Hyperbolic functions** available from https://en.wikipedia.org/wiki/Hyperbolic_functions

Note that tanh takes values in $(-1, 1)$, so this polynomial approximates $\texttt{sign}_{-1,1,0}(x) = -1 + 2\,\texttt{sign}(x)$ rather than our 0/1 sign function. Solving for $\texttt{sign}(x)$, we use the rescaled version

$$\texttt{sign}(x) \approx \frac{1 + \tanh(x)}{2} \approx \frac{1}{2} + \frac{x}{2} - \frac{x^3}{6} + \frac{x^5}{15} - \frac{17x^7}{630} + \frac{31x^9}{2835}.$$

The halving and shift can be folded into the coefficients, so this costs nothing beyond the tanh polynomial. It's also just the logistic polynomial evaluated at $2x$, since $\frac{1+\tanh(x)}{2} = \sigma(2x)$. That's where the radius of $\pi/2$ comes from: the logistic series converges for $|2x| < \pi$.

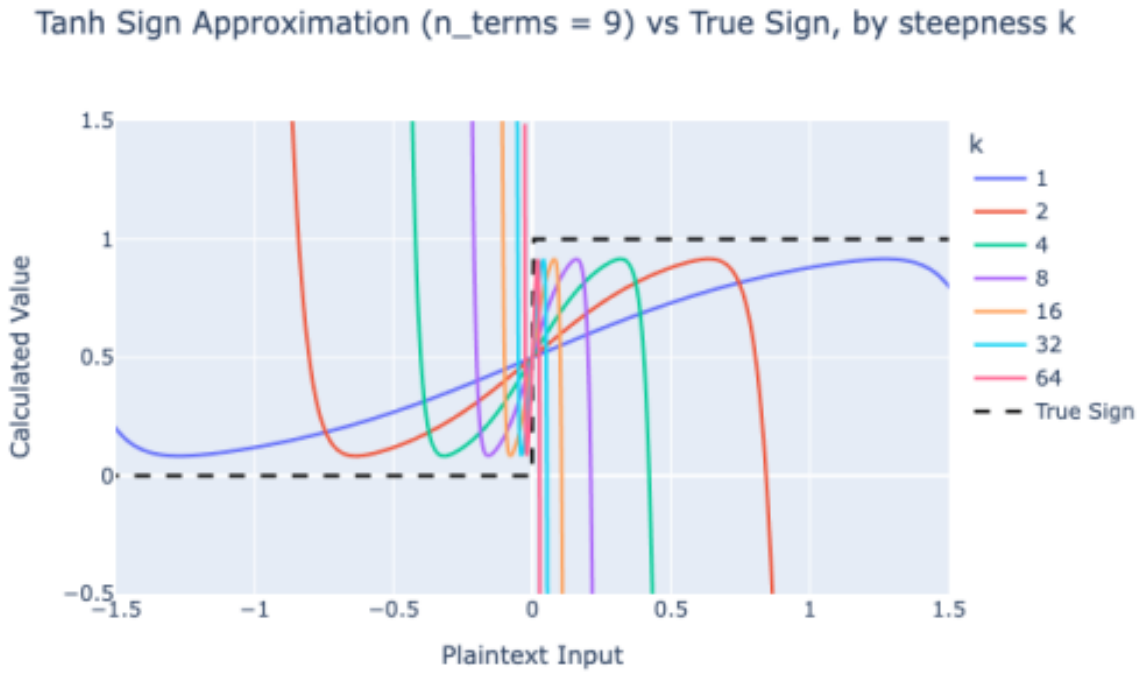



**Figure**: different graphs of the function $\tanh(kx)$ with steepness parameter $k$.

# 14 Sign Applicability

Here we provide the windows in which our (naive) implementation of sign (via logistic function approximation) has an error of 10% or less. Error is calculated based off of a target of 0 or 1. The target is set to 0 for an input $x < \frac{1}{2}$ and 1 for input $x > \frac{1}{2}$. The error is the absolute difference between the target and approximate values expressed as a percentage.

| k | Left band | Right band | Window width |
|---|---|---|---|
| 1 | [-2.543, -2.210] | [2.210, 2.543] | 0.333 |
| 2 | [-1.272, -1.105] | [1.105, 1.272] | 0.167 |
| 4 | [-0.636, -0.553] | [0.553, 0.636] | 0.083 |
| 8 | [-0.318, -0.277] | [0.277, 0.318] | 0.041 |
| 16 | [-0.159, -0.139] | [0.139, 0.159] | 0.020 |
| 32 | [-0.079, -0.070] | [0.070, 0.079] | 0.009 |
| 64 | [-0.040, -0.035] | [0.035, 0.040] | 0.005 |

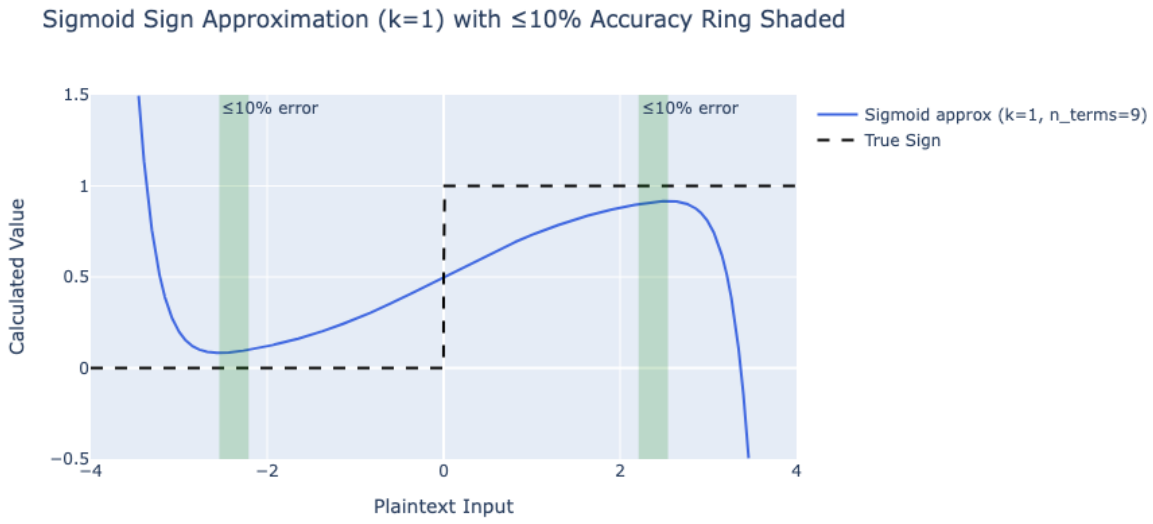



Figure 14.1: sigmoid_accuracy_ring

Now we present the analogous accuracy windows for hyperbolic tangent as previously described for the logistic function. Error is calculated in the same manner.

It is worth noting that these accuracy windows are about half the size compared to those of the logistic function. This is due to the radii of convergence being $\pi/2$ and $\pi$ for hyperbolic tangent and logistic respectively. Hence, hyperbolic tangent is naturally steeper than logistic and approximates the real value faster on a similar interval.

| k | Left band | Right band | Window width |
|---|---|---|---|
| 1 | [-1.272, -1.105] | [1.105, 1.272] | 0.167 |
| 2 | [-0.636, -0.553] | [0.553, 0.636] | 0.083 |
| 4 | [-0.318, -0.277] | [0.277, 0.318] | 0.041 |
| 8 | [-0.159, -0.139] | [0.139, 0.159] | 0.020 |
| 16 | [-0.079, -0.070] | [0.070, 0.079] | 0.009 |
| 32 | [-0.040, -0.035] | [0.035, 0.040] | 0.005 |
| 64 | [-0.020, -0.018] | [0.018, 0.020] | 0.002 |

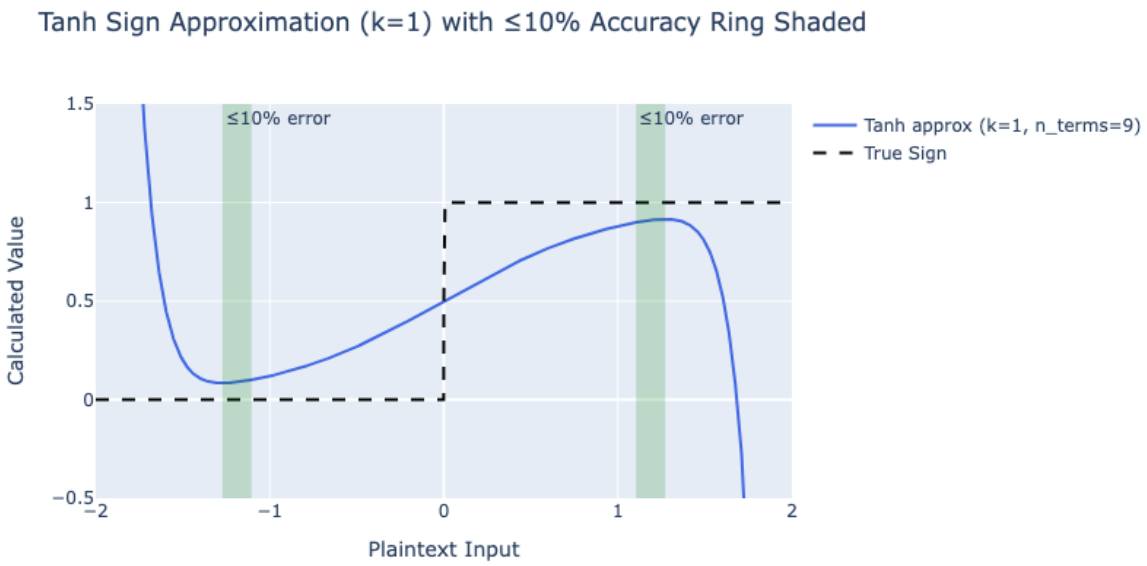



Figure 14.2: tanh_accuracy_ring

**Remark**

We can see a higher $k$ compresses the interval of high accuracy towards 0. We note that given the right compression method for a dataset we could approximate a reasonable sign value for the whole dataset. Requiring a user input of known maximum and minimum value of the input (beyond $(-\infty, \infty)$ ) would support this scaling. [⁀prantl]

Within our current experimentation we found the window of high accuracy convergence scaled in proportion to the window of convergence, roughly 15% of the observed data.

### 14.0.1 Inequality and Equality

Inequality and sign functions are essentially equivalent. If we have an useful sign function, we can effectively do an "inequality" function as follows:

$$(x > a) := \texttt{sign}_{0,1,a}(x),$$

which returns 1 for each element $x_i$ of the vector $x$ if $x_i > a$ or 0 otherwise. Of course, problems may happen very close to $a$, so one must be careful while using it in the construction of other operations.

With this, we can do an "equality" function. Because we are doing floating point arithmetic, it is inadvisable to do a "true equality" check, so what we mean when we say "$x$ is equal to $a$" means that their difference is small (up to an error $\epsilon$). This can be done via

$$(x == a) := (x > a - \epsilon) * (x < a + \epsilon),$$

which gives 1 exactly when both conditions are met; that is, $x$ is within $\epsilon$ of $a$ and 0 otherwise. As an example,

$$(x_i == 0) := (x_i \geq -0.01) * (x_i \leq 0.01)$$

obtains a 0-1 vector, which has 1 in place $i$ if and only if $x_i$ is close (within 0.01) to 0.

# Part V

# Statistics

This section introduces the application of statistical functions, first through moments and the sample mean. We introduce the idea of the difference matrix, which builds into our implementation of median and mode. We then move to the correlation and regression coefficients, which then extends to simple and multiple linear regression.

In a fully homomorphic environment, standard inference is thrown out of the window. It is important to note that in our case, computations accumulate noise simply from being performed in an FHE environment. It is also notable that many of the methods in this section depend on stable implementations of square root and division. Both of these are built through the use of floating point estimation, which suffer from additional errors.

# 15 Higher Moments

Using the sum technique and repeated squaring (on multiplication), we are able to compute higher moments fairly efficiently.

Let vector $\vec{x}$ be an encoded series of $n$ sample measurements):

$$\vec{x} = (x_1, x_2, \dots, x_n)$$

Define the $k^{th}$ **(raw) population moment** to be the expectation of the $k^{th}$ power of the random $X$ and the $k^{th}$ **(raw) sample moment** to be its estimator.

$$E[X^k] = \mu'_k = E[m_k]$$
$$m_k = \frac{1}{n}(x_1^k + x_2^k + \cdots + x_n^k).$$

In order to obtain the $k^{th}$ sample moment, we must perform three actions:

1. Compute $(x_1^k, x_2^k, \dots, x_n^k)$.
2. Compute $(x_1^k + x_2^k + \dots + x_n^k)$.

Performing the intra-vector sum operation allows us to get the vector $(x_1^k + \cdots + x_n^k) \cdot (1, 1, \cdots, 1)$ in $O(\log(n))$ additional operations.

3. Divide by $n$ (a known constant) by multiplying by the vector $(1/n) \cdot (1, 1, \cdots, 1)$.

Naturally, the first sample moment only needs steps 2 and 3:

$$m_1 = \bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

We are then able to compute the following variations:

1. **Central moment**: The $k^{th}$ population central moment of $X$ about mean $\mu$ is

$$E[(X-\mu)^k] = E[(X-E[X])^k].$$

The $k^{th}$ sample central moment of $X$ about mean $\mu$ is

$$m_k = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^k$$

Note: The $2^{nd}$ central moment is used to calculate the variance:

$$\texttt{var}(X) = E[(X-E[X])^2] = E[X^2] - \mu^2.$$

2. **Standardized moment**: This is the population central moment but $X-\mu$ is scaled by the standard deviation $\sigma$ before exponentiation:

$$E\left[\left(\frac{X-\mu}{\sigma}\right)^k\right].$$

The sample standardized moment is then

$$\frac{m_k}{s^k} = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i-\bar{x})^k}{\sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i-\bar{x})^2}^k}$$

**Remark**

1. We can expensively calculate $\sqrt{\texttt{var}(X)} = \sigma$, the standard deviation, or pass this operation back to the user to help find the standardized moment. (see **square roots** later)
2. Note: You will see that ciphertext division is notably more difficult than scalar division, but we can also rely on user input/output when standardizing.
3. With this section, we can use the method of moments to find hidden distribution parameters.

# 16 Difference Matrix Method

A standard triad of summary statistics is the mean, median, and mode. The mean is a natural consequence of summations and moments. For the median and mode, the naive methods require branching logic, which we don't have. We introduce what we call the **difference matrix method** which will allow us to implement both statistics in a similar way (though still requiring tools from the Discrete Math section, especially Equality and Max).

The tech tree below shows how the difference matrix method fits together, read from left to right. The basic moves build the difference matrix and sign. Sign gives equality and max, and the difference matrix and equality then combine into the median and the mode, with max also feeding the mode.

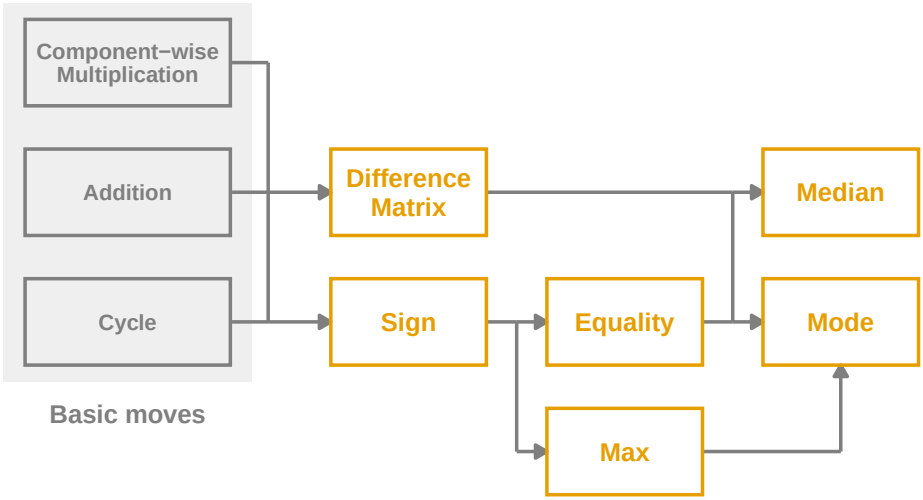



Figure 16.1: The tech tree for this section. The basic moves in the grey band build up to the difference matrix method, shown in bold orange.

The method is as follows:

Given a vector $z = (z_1, \ldots, z_{N/2})$, we construct an $(N/2) \times (N/2)$ matrix $D$ where $D_{i,j} = z_i - z_j$. We will call our matrix the **difference matrix.**

**Info**

**Remark**: our method is general enough that we should be able to compute $D$ where $D_{ij} = f(z_i, z_j)$ for some function $f$. We mention this but do not use the more general method in this work.

Here is how we would generate the difference matrix:

1. Assuming we are in the inter-vectors case, we can assume the $z_i$ are given as $u^{(i)} = (0, \dots, 0, z_i, 0, \dots, 0)$.
2. We can "broadcast" the $z_i$ into every entry by computing

$$v^{(i)} := \texttt{sum}(u^{(i)}) = (z_i, \dots, z_i).$$

3. Now we can separately compute:
   - $d_i = v^{(i)} - z = (z_i - z_1, z_i - z_2, \dots, z_i - z_{N/2})$ for all $i$. This is row $i$ of $D$.
   - If we wanted the transpose instead (column $i$ of $D$), we can compute: $d'_i = z - v^{(i)} = (z_1 - z_i, z_2 - z_i, \dots, z_{N/2} - z_i)$. Since $D_{j,i} = -D_{i,j}$, this is just $-d_i$.
4. We can think of the matrix as the combination of these vectors, though we might also want to isolate the use of our $N/2$ "difference vectors" of the form

$$d_i = (d_{i,1}, \dots, d_{i,N/2}), \qquad d_{i,j} = D_{i,j} = z_i - z_j.$$

**Notation**

In the Mode and Median sections, $z_i$ always denotes a single input number, while $d_i$ denotes a whole row of $D$, stored as one vector and processed element-wise.

**Info**

**Remark**: the above assumes the inter-vectors case. If we wanted the intra-vectors case, we can do something like the following:

1. Given an input vector $z = (z_1, \dots, z_{N/2})$,
2. Define indicator vectors $c_i = (0, 0, \dots, 1_i, 0 \dots, 0)$.
3. Compute $v^{(i)} = \texttt{sum}(c_i * z) = (z_i, \dots, z_i)$; the product $c_i * z$ isolates the value of $z$ at index $i$, and the intravector sum broadcasts it to every entry,

…and then the rest is the same as before.

# 17 Mode

Obtaining the mode should be a bit easier for discrete values (similar to bucketing), so we approach the continuous case, given a vector $z = (z_1, \dots, z_{N/2})$ with assumed real values for simplicity.

Here we compute the difference matrix $D$ for $z$, with entries $d_{i,j} = z_i - z_j$, which results in a matrix stored as the row vectors $d_1, \dots, d_{N/2}$. Row $d_i$ holds the differences between $z_i$ and every entry of $z$.

| | $z_1$ | $z_2$ | $z_3$ | $\cdots$ | $z_{N/2}$ |
|---|---|---|---|---|---|
| $d_1$ | $0$ | $d_{1,2}$ | $d_{1,3}$ | $\cdots$ | $d_{1,N/2}$ |
| $d_2$ | $d_{2,1}$ | $0$ | $d_{2,3}$ | $\cdots$ | $d_{2,N/2}$ |
| $d_3$ | $d_{3,1}$ | $d_{3,2}$ | $0$ | $\cdots$ | $d_{3,N/2}$ |
| $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\ddots$ | $\vdots$ |
| $d_{N/2}$ | $d_{N/2,1}$ | $d_{N/2,2}$ | $d_{N/2,3}$ | $\cdots$ | $0$ |

Note: Some of the non-diagonal elements $d_{i,j}$ can also be 0, namely whenever $z_j = z_i$. For instance, we expect the most nondiagonal zeros in the row $d_i$ of the mode $z_i$.

Now, the idea here is to check every row $d_i$ and then look for the one with the most number of (approximate) zeros. This will correspond to the mode. Specifically,

1. For every row $d_i$, we can compute

   $$y_i := (d_i == 0)$$

   on it (element-wise) to get a 0-1 vector $y_i$ that has a number of 1s equal to the number of terms approximately equal to $z_i$ (including $z_i$ itself, from the diagonal).

2. Now we compute the intravector sum

   $$s_i = \texttt{sum}(y_i)$$

   to get these counts.

3. Computing a max over this would get the mode.

# 18 Median

We can employ a similar method to find the median value. For this method, we assume for simplicity that all the numbers are distinct and $N/2$ is odd.

1. Create a matrix $D^{\text{sign}}$ with entries

$$d_{i,j}^{\text{sign}} = \texttt{sign}_{-1,1,0}(z_i - z_j).$$

   We can just compute the same difference matrix $D$ and then run the sign function over every row, giving the rows $d_i^{\text{sign}} := \texttt{sign}_{-1,1,0}(d_i)$. This looks like:

| | $z_1$ | $z_2$ | $z_3$ | $\cdots$ | $z_{N/2}$ |
|---|---|---|---|---|---|
| $d_1^{\text{sign}}$ | 0 | $-1$ | $-1$ | $\cdots$ | $-1$ |
| $d_2^{\text{sign}}$ | 1 | 0 | $-1$ | $\cdots$ | $-1$ |
| $d_3^{\text{sign}}$ | 1 | 1 | 0 | $\cdots$ | $-1$ |
| $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\ddots$ | $\vdots$ |
| $d_{N/2}^{\text{sign}}$ | 1 | 1 | 1 | $\cdots$ | 0 |

though unlike $D$ from earlier, $d_{i,j}^{\text{sign}} \in \{-1, 1\}$ off the diagonal.[1]

The example below works through $z = (7, 2, 9, 4, 5)$, with the row of the median in orange.

$$\underset{D^{\text{sign}}}{\begin{bmatrix} 0 & 1 & -1 & 1 & 1 \\ -1 & 0 & -1 & -1 & -1 \\ 1 & 1 & 0 & 1 & 1 \\ -1 & 1 & -1 & 0 & -1 \\ \mathbf{-1} & \mathbf{1} & \mathbf{-1} & \mathbf{1} & \mathbf{0} \end{bmatrix}} \xrightarrow{\text{row sums}} \underset{s}{\begin{bmatrix} 2 \\ -4 \\ 4 \\ -2 \\ \mathbf{0} \end{bmatrix}} \xrightarrow{==0} \underset{m}{\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \mathbf{1} \end{bmatrix}}$$

**Step 1.** Build $d_{i,j}^{\text{sign}} = \texttt{sign}_{-1,1,0}(z_i - z_j)$ with 0 on the diagonal, so row $i$ counts $+1$ for each value below $z_i$ and $-1$ for each value above it.

**Step 2.** Summing each row gives $s = (2, -4, 4, -2, 0)$, and only the row of $z_5 = 5$ sums to 0, so $m = (0, 0, 0, 0, 1)$ and $\texttt{sum}(m * z) = 5$, the median.

[1] Landon Hurley; **Completing and studentising Spearman's correlation in the presence of ties** available from https://arxiv.org/pdf/2512.23993

2. Now, if $z_i$ were the median, then it would have an equal number of $-1$s and 1s in its row, which is equivalent with having row sum 0 (note that this implicitly assumes that there is no multiplicity of the median; more accurately, the median occurs an odd number of times).

3. Thus, we can add the rows $d_i^{\text{sign}}$ element-wise and then compute equality with zero:

$$m := ((d_1^{\text{sign}} + d_2^{\text{sign}} + \cdots + d_{N/2}^{\text{sign}}) == 0)$$

   to obtain a 0-1 vector that points out which *index* the median falls in.

4. If we are interested in the *value* of the median instead of the *index,* then we can just multiply $\texttt{sum}(m * z)$. Note that this sum only works if exactly one element is the median: if, say, two elements are the median, then we would get twice the median if we sum.

**Remarks**

1. If we want to sort for any reason, then getting the median from the sort is immediate and we would not need to add extra computation.

2. Constructing quartiles is similar to the median. If we use the "sum sign functions" method, then getting a 75% quartile is just finding something with row sum $N/2$ instead of 0.

Pushing onward, we now extend from univariate summaries to measures of dependence, such as correlation and related functions, which capture relationships between variables.

# 19 Two Coefficients

Given vectors $\mathbf{x}$ and $\mathbf{y}$, there are 2 closely related quantities:

1. The **correlation coefficient**[1], a measure of normalized linear association, is
$$r := r(x,y) = \frac{\sum_{i=1}^{N/2}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{N/2}(x_i - \bar{x})^2 \sum_{i=1}^{N/2}(y_i - \bar{y})^2}}.$$
2. Often people are instead interested in the square $r^2$, which is called the **coefficient of determination**[2].
3. The **regression coefficient**[3], a measure of linear rate of change, is
$$\beta = \frac{\sum_{i=1}^{N/2}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{N/2}(x_i - \bar{x})^2}$$

These can be approached with the same general strategy:

1. Compute the means $\bar{x}$ and $\bar{y}$ and subtract them off from the original vectors;
2. Compute the numerators and denominators separately;
3. Divide (which might be costly).

With some more detail:

Given $x$ and $y$, we can compute vectors that contains their means: $x_m = (\bar{x}, \bar{x}, \cdots)$ and $y_m(\bar{y}, \bar{y}, \cdots)$. Then we can subtract to obtain

$$x' = (x_1, x_2, \dots, x_{N/2}) + (\bar{x}) * (-1, -1, \dots, -1) \tag{19.1}$$
$$= (x_1 - \bar{x}, x_2 - \bar{x}, \dots, x_{N/2} - \bar{x}) \tag{19.2}$$

and similarly

$$y' = (y_1 - \bar{y}, y_2 - \bar{y}, \dots, y_{N/2} - \bar{y}) \tag{19.3}$$

[1] **Correlation coefficient** available from https://en.wikipedia.org/wiki/Correlation_coefficient
[2] **Coefficient of determination** available from https://en.wikipedia.org/wiki/Coefficient_of_determination
[3] **Linear regression** available from https://en.wikipedia.org/wiki/Linear_regression

Our task for $r$ then (the work for the regression coefficient is similar) becomes computing

$$r(x', y') = \frac{\sum_{i=1}^{N/2}(x'_i)(y'_i)}{\sqrt{\sum_{i=1}^{N/2}(x'_i)^2 \sum_{i=1}^{N/2}(y'_i)^2}}$$

1. The numerator is the intra-vector sum of $(x') * (y')$.
2. The denominator is the square of the (scalar) product of two intravector sums of $(x')^2$ and $(y')^2$.
3. Finally, we need to do a division.

**Remark**

As usual (for the correlation coefficient; the regression coefficient is similar), we can avoid the possibly costly division and square roots for $r$ by giving the user the numerator and the *square* of the denominator separately and have the user compute the coefficient. If we are willing to do the division but want to save a square root, we can just give them $r^2$ instead.

# 20 Linear Regression

### 20.0.1 Simple Linear Regression

Linear regression aims to find the line of best fit for a given set of data points. Recall the method of least squares,

$$Y = \beta_0 + \beta_1 x + \epsilon$$

where $\beta_0$ and $\beta_1$ are the regression coefficients of slope and intercept, Y is our response variable, x is our independent variable, and $\epsilon$ is our error, which now in FHE, includes additional computation error.

We assume a simple protocol where:

1. Take as client input $x = (x_1, \dots, x_n)$ and $y = (y_1, \dots, y_n)$
2. Output a model
$$\hat{y} = \hat{\beta}_1 x + \hat{\beta}_0$$
that forms the line of best fit (slope and intercept) for our given client input and can additionally be input any new independent variable to generate predictions along the newly generated line.

These are immediate from our work:

1. We can first compute $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$ and $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$, which we can do since they are **means**.
2. We can compute the slope coefficient
$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$
3. The result is then easily used to find the intercept
$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$
using a multiplication and a subtraction.

### 20.0.2 (Sketch of) Multiple Linear Regression

**Multiple linear regression** extends the prior one-dimensional context to allow for each data point to take multiple input features/ predictors. Recall the model for multiple linear regression:

$$\mathbf{Y} = \mathbf{X}\beta + \epsilon$$

A natural protocol is:

1. Take as client input

$$\mathbf{Y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1k} \\ 1 & x_{21} & x_{22} & \dots & x_{2k} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{nk} \end{bmatrix}$$

   where the $y_i$'s are the responses and each row of the $\mathbf{X}$ matrix is one observation's independent variables.

2. Output a model

$$\hat{\mathbf{Y}} = \mathbf{X}\hat{\beta}$$

   that forms the line of best fit for our given client input and can additionally be input a set of new independent variables to generate predictions along the newly generated line.

Our desired weight vector $\beta$ can be computed with

$$\hat{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$$

The underlying parts, however, need more work. First, this approach requires matrix operations, of which our explorations are very preliminary (see Dot Products and Matrix Multiplication). Most crucially, the central difficulty is that we would need to invert an $m \times m$ matrix; at a quick glance, we can inherently approach this similarly to division using Newton's method, as

$$(I - M)^{-1} = I + M + M^2 + \cdots,$$

but there are still many moving parts under the hood.

**Remark**

See [1] for some independent work on matrix inversion under FHE.

[1] Seunghu Kim, Seongbong Choi, Hyung Tae Lee; **Improved Matrix Inversion with Packed Ciphertexts using Fully Homomorphic Encryption** available from https://eprint.iacr.org/2025/1274.pdf

# Part VI

# Sorting

Sorting an encrypted vector can provide useful information, including its minimum, maximum, median, and quartiles. But ordinary sorting algorithms often branch on comparison results or use encrypted values as memory indices, operations that are unavailable in FHE. We instead must attempt to sort using only arithmetic operations on encrypted values.

We will compare methods by their total comparisons, sequential comparison layers, and arithmetic depth. The chapters develop these ideas in order: PairSort and bubble sort, parallel schedules, rank-based sorting, then maximum extraction. The discrete-model section gives a comparison construction for bounded integer inputs; here the focus is how sorting methods use comparisons.

## Comparison Based Sorting

The first intuitive method is to utilize sorts that compare and order pairs of values on a schedule irrelevant of the input of the data. Sorts such as merge and quicksort are not suitable for FHE because their branching depends on the results of the comparison. In FHE, comparisons can be expressed as an arithmetic operation using a polynomial approximation of the sign function.

We call this arithmetic operation on a pair of values PairSort, defined in the bubble sort chapter. It can be used to emulate sorts prevalent in sorting networks, which allow for a fixed schedule of comparisons regardless of the results.

## Rank and place

A different strategy exists that avoids moving values through successive swaps. It compares each pair, counts how many values are smaller than each input, then places each value at its rank. This uses fewer independent comparisons, as all the comparisons are independent and can be computed in parallel. However, this requires for a ciphertext $n$, the ability to compute on a vector of size $n^2$. It can be preferable when the input suits the size constraint, as it can allow for less comparison depth than swap based methods. One can return all the ranks of the inputs as its own result, but taking this encrypted rank and using it to place the indexes in their sorted positions is also considered.

## Maximum and minimum

A consequence of sorting is that the maximum and minimum values are available at the end of the sort. However, we can also compute the maximum and minimum in $O(\log(n))$ time using a tournament style comparison, which is more efficient than sorting.

# 21 Bubble Sort

As a proof-of-concept, we choose to use **bubble sort**[1] as a starting point for sorting.
Despite being a well-known "slow" sorting algorithm, it is a good candidate for FHE prototyping because its operations are local: only swaps, with no dynamic memory allocation (which e.g. quicksort would need). Each of its steps is one operation on a pair of positions, so we define that operation first.

### 21.0.1 PairSort

Given an $n$-length vector $x$, the following move will return an $n$-length vector $y$ that is equal to $x$ in every coordinate except possibly $x_i$ and $x_j$.
We call this move $\texttt{PairSort}_{i,j}(x)$:

> Compare $x_i, x_j$ (typically, $i + 1 = j$).
> (1) If $x_i < x_j$, then $y = x$ with no swapping performed.
> (2) If $x_i > x_j$, then $y$ has $x_i$ and $x_j$ swapped.

PairSort is known elsewhere as a compare-and-swap or comparator. To perform it without branching in FHE, we use a polynomial approximation of the sign function.

For two encrypted values $x_i$ and $x_j$, let

$$s = \texttt{sign}(x_i - x_j),$$

where the sign function is approximated by a polynomial and returns approximately 1 when $x_i \geq x_j$ and 0 when $x_i < x_j$. This replaces an ordinary conditional such as "if $x_i \geq x_j$, swap" with arithmetic selection:

$$y_i = sx_j + (1 - s)x_i,$$
$$y_j = sx_i + (1 - s)x_j.$$

This gives $y_i$ the smaller value between $\{x_i, x_j\}$ and $y_j$ the larger value.

For $(x_i, x_j)$, the outputs are computed to be $(y_i, y_j)$.

$(x_i, x_j) = (7, 3)$, $s \approx 1$ so the output is $(3, 7)$.

[1] **Bubble sort** available from https://en.wikipedia.org/wiki/Bubble_sort

$(x_i, x_j) = (3, 7)$, $s \approx 0$ so they remain $(3, 7)$.

The same operations run in both cases; no branch depends on encrypted data. The approximation is reliable only on its intended input range and away from the transition near equality.

We treat PairSort as an abstraction that we assume we have. The construction above is only a suggested implementation, since its approximate comparison runs into floating point issues. The sorts in these chapters are therefore proofs of concept that show what a reliable PairSort would make possible. We expect this to be an implementation problem, not a fundamental barrier.

It's a bit tricky to actually implement this with vector-wide operations without being wasteful. Here is one draft:

1. Construct constant 1-hot vectors $c_1 = (1, 0, \ldots, 0)$, $c_2 = (0, 1, 0, \ldots)$, … Let $x = (x_1, \ldots, x_n)$.
2. For a given $i, j$, compute

$$m' = (c_i - c_j) * (x_i, x_i, \ldots, x_i) + (c_j - c_i) * (x_j, x_j, \ldots, x_j).$$

   This will have a 0 everywhere, except $m'_i = x_i - x_j$ and $m'_j = x_j - x_i$.
3. Now multiply $m = (c_i - c_j) * m'$. This will also have a 0 everywhere, except $m_i = m_j = x_i - x_j$.
4. Let $s = -\texttt{sign}(m) * m'$. Here,
   - $s_i$ will be $x_j - x_i$ if $x_i > x_j$, 0 otherwise.
   - $s_j$ will be $x_i - x_j$ if $x_i > x_j$, 0 otherwise.
   - all other elements are 0.
5. Compute $y = x + s$. If $x_i < x_j$ then $s_k = 0$ everywhere, so $y = x$. If $x_i > x_j$ then $y_i = x_i + (x_j - x_i) = x_j$, and $y_j = x_j + (x_i - x_j) = x_i$. In other words, the result flips $x_i$ and $x_j$ if $x_i > x_j$, meaning the smaller element goes to $x_i$ and the larger element goes to $x_j$.

The resulting $y$ is $\texttt{PairSort}_{i,j}(x)$. Note this is fully FHE legal and involves no branching.

### 21.0.2 The Bubble Sort Algorithm

Bubble sort passes through a given vector, applying PairSort to each neighboring pair. Each pass moves the largest remaining element to the end of the unsorted part, which takes $O(n^2)$ repeated pairwise element comparisons to finish the sort:

1. Perform $\texttt{PairSort}_{1,2}(x)$, …, $\texttt{PairSort}_{n-1,n}(x)$. These $n-1$ moves ensure the maximum element is now at $x_n$.
2. Perform $\texttt{PairSort}_{1,2}(x)$, …, $\texttt{PairSort}_{n-2,n-1}(x)$. These $n-2$ moves ensure the second-largest element is now at $x_{n-1}$.

3. …

```
NULL
```

$O(n^2)$ total comparisons (with $O(n)$ per pass) will entirely sort $x$. However, this gets fairly costly and noisy due to the multiplications in the approximated comparisons.

Figure 21.1 shows the complete schedule for eight inputs. Each horizontal line is a vector position and each arrow is one PairSort, with the larger value moving toward the arrowhead.[2]

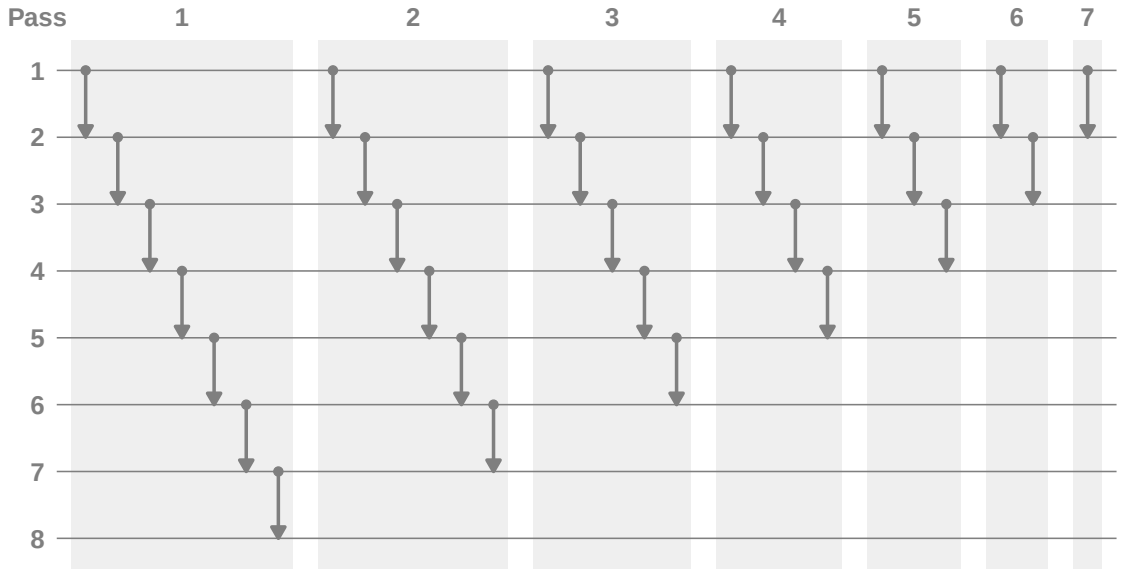



Figure 21.1: The bubble sort network for eight inputs, with 7 passes, 28 layers, and 28 PairSorts. Only one PairSort runs in each layer.

[2]Each layer (a column of arrows) is one batched sign evaluation, a single SIMD step, so the number of layers is the number of sequential comparison steps. Every layer holds a single PairSort, so this network needs 28 sequential comparison steps for 8 inputs.

# 22 Odd-Even Sort

Bubble sort applies PairSort to two elements in a vector, then uses its updated vector in the next comparison. In FHE we have a unique advantage, that being the vectorwide nature of computation. This causes computations batched on a ciphertext to be functionally the same cost regardless of how much of the ciphertext is populated. Using this, we can organize the comparisons into layers of adjacent pairs, where each layer is a single vectorwide PairSort. This is called odd-even sort. It is a simple example of optimizations that can be made to bubble sort, and is a stepping stone to more advanced sorting networks.

Odd-even sort organizes the comparisons into two alternating layers [1]. Number the vector positions from 1. The **odd layer** compares

$$(1,2),\ (3,4),\ (5,6),\dots,$$

and the **even layer** compares

$$(2,3),\ (4,5),\ (6,7),\dots.$$

For one iteration, let the input be $x = (x_1,\dots,x_N)$ with $N$ even. In the odd layer, apply PairSort to the disjoint pairs $(x_1,x_2)$, $(x_3,x_4),\dots,(x_{N-1},x_N)$ simultaneously. Write the resulting vector as

$$x' = (a_1,b_1,a_2,b_2,\dots,a_{N/2},b_{N/2}),$$

where $(a_k,b_k) = (\min(x_{2k-1},x_{2k}),\max(x_{2k-1},x_{2k}))$ for $k = 1,\dots,N/2$. Then the even layer applies PairSort to $(b_1,a_2),(b_2,a_3),\dots,(b_{N/2-1},a_{N/2})$ simultaneously, leaving the endpoints in place. This produces the next vector

$$x'' = (a_1,c_1,d_1,c_2,d_2,\dots,c_{N/2-1},d_{N/2-1},b_{N/2}),$$

[1] N. Habermann, **Parallel neighbor-sort (or the glory of the induction principle)**, available from https://kilthub.cmu.edu/articles/journal_contribution/Parallel_neighbor-sort_or_the_glory_of_the_induction_principle_/6608258?file=12099395

where $(c_k, d_k) = (\min(b_k, a_{k+1}), \max(b_k, a_{k+1}))$ for $k = 1, \ldots, N/2 - 1$. One odd layer followed by one even layer is one odd-even iteration; repeating this fixed schedule moves values toward their sorted positions.

Repeat the alternating layers for $N$ phases to sort $N$ inputs. Sequentially, bubble sort performs $O(N^2)$ individual PairSorts, but Odd-even sort instead needs only $O(N)$ **comparison layers**, due to only requiring one vector-wide comparison-polynomial evaluation per pass, regardless of how many pairs are being compared.

Figure 22.1 shows the complete schedule for eight inputs. Each horizontal line is a vector position and each arrow is one PairSort, with the larger value moving toward the arrowhead.[2]

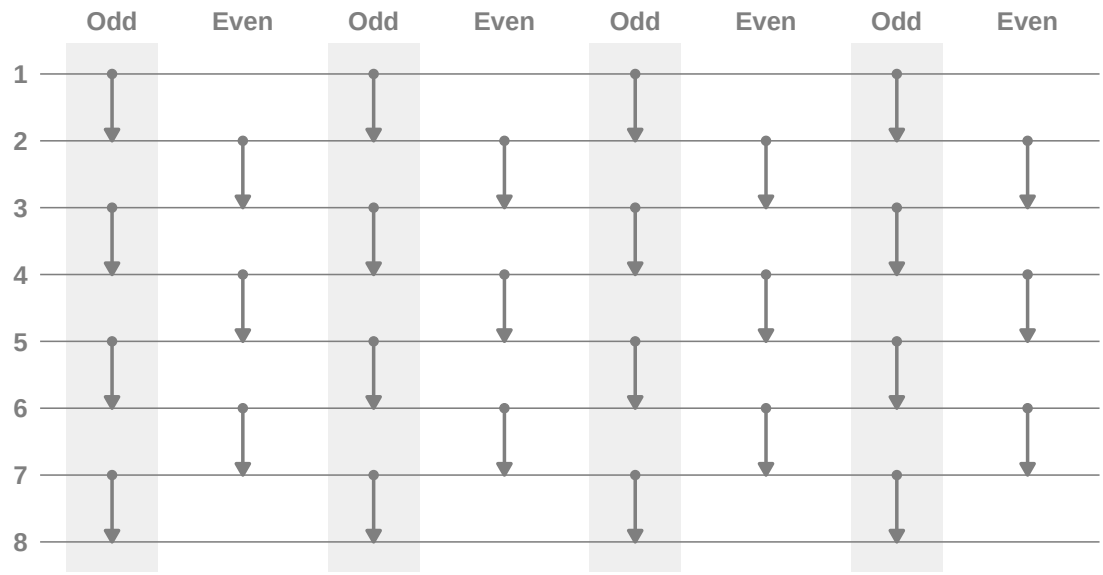



Figure 22.1: The odd-even sorting network for eight inputs, with 8 layers and 28 PairSorts.

For the PairSort operation used in each layer, see Section 21.0.1.

[2]Each layer (a column of arrows) is one batched sign evaluation, a single SIMD step, so the number of layers is the number of sequential comparison steps. This network has 8 layers of 4 or 3 disjoint PairSorts, so it needs 8 sequential comparison steps for 8 inputs.

# 23 Bitonic Sort

Odd-even sort reduces depth by comparing many neighboring pairs at once. But its comparisons only move values one position per round. Values that start far apart must make progress through many neighboring comparisons, so an $N$-element input can require $N$ rounds and $O(N^2)$ PairSorts. In FHE, where each comparison layer has a cost, this leaves room to improve.

A **sorting network** is a fixed schedule of PairSort operations. Each layer contains disjoint pairs, so all comparisons in that layer can run in parallel. Unlike an ordinary comparison sort, the schedule does not depend on the input values. The design challenge is to choose a schedule that moves values into order with fewer layers and fewer total comparisons.

#### 23.0.0.1 Bitonic sequences and merging

Bitonic sort builds and merges **bitonic sequences** [6], which are sequences that are first increasing and then decreasing. A bitonic merge compares elements that are far apart first, then progressively compares closer pairs. This lets large values move toward the end and small values toward the beginning in a few structured layers, rather than only one position at a time. To sort a block, we sort its first half in ascending order and its second half in descending order, which makes the whole block bitonic, then merge it, repeating this from pairs up to the full vector. The result is a sorting network with $O(\log^2 N)$ layers and $O(N \log^2 N)$ total comparisons [6], which is better than odd-even sort's $O(N)$ layers and $O(N^2)$ comparisons.

#### 23.0.0.2 Merging a Bitonic Sequence

Below is an example of a bitonic merge on a sequence of eight values, starting from a bitonic sequence. In practice this sequence is built from smaller bitonic sequences, but the merge works the same way. The input is

$$(2, 5, 7, 8, 6, 4, 3, 1).$$

An ascending bitonic merge starts by comparing positions 1 and 5, 2 and 6, 3 and 7, and 4 and 8, placing the smaller value in the first position of each pair. In the steps below, values with the same color are compared with each other, and the positions compared are listed above the arrow:

$$(2,5,7,8,6,4,3,1) \xrightarrow{(1,5),\ (2,6),\ (3,7),\ (4,8)} (2,4,3,1,6,5,7,8).$$

The two halves are now bitonic subsequences, as long as we also count rotations of a bitonic sequence: $(6,5,7,8)$ is a rotation of $(7,8,6,5)$, and the merge works the same way on it. Repeat the merge within each half, now comparing positions two apart:

$$(2,4,3,1,6,5,7,8) \xrightarrow{(1,3),\ (2,4),\ (5,7),\ (6,8)} (2,1,3,4,6,5,7,8),$$

then compare adjacent pairs:

$$(2,1,3,4,6,5,7,8) \xrightarrow{(1,2),\ (3,4),\ (5,6),\ (7,8)} (1,2,3,4,5,6,7,8).$$

Those three layers merge one bitonic sequence of length eight. Bitonic sort constructs larger bitonic sequences from smaller ones, then applies these merges. For $N$ a power of two, its network has $\frac{\log_2 N(\log_2 N+1)}{2}$ layers [6].

Figure 23.1 shows the complete schedule for eight inputs. Each horizontal line is a vector position and each arrow is one PairSort, with the larger value moving toward the arrowhead.[1]

#### 23.0.0.3 How much does this save?

For eight inputs, odd-even sort uses eight alternating odd/even comparison rounds. Each round compares three or four neighboring pairs, for 28 PairSorts total. The bitonic network uses six layers and 24 PairSorts. Both schedules can batch each layer's disjoint comparisons across ciphertext slots, but bitonic sort needs fewer sequential layers and fewer comparisons in this example. Since each layer is one vector-wide evaluation, the layer count is the saving that matters most in FHE. Bitonic sort functionally uses $O(\log^2 N)$ layers and $O(N \log^2 N)$ comparisons [6], versus $O(N)$ rounds and $O(N^2)$ comparisons for odd-even sort.

Each layer still uses the same PairSort operation described in Section 21.0.1. The next step is to look at a different way to organize sorting: compute how many values in the array are less than a given value, then move it to that index.

[1] Each layer (a column of arrows, drawn side by side when arrows would otherwise overlap) is one batched sign evaluation, a single SIMD step, so the number of layers is the number of sequential comparison steps. This network has 6 layers, so it needs 6 sequential comparison steps for 8 inputs.

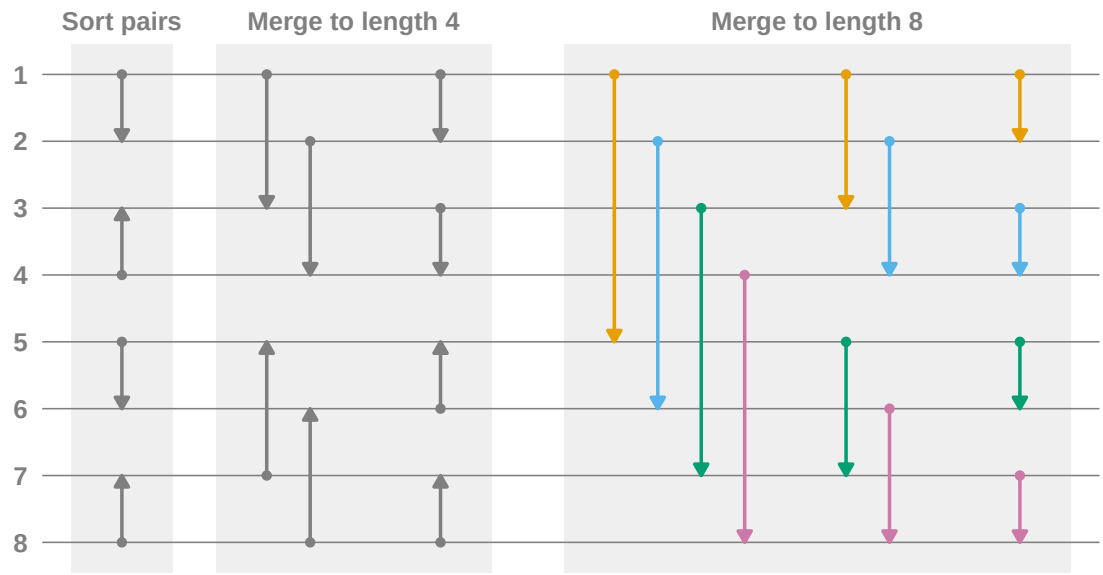



Figure 23.1: The bitonic sorting network for eight inputs, with 6 layers and 24 PairSorts. The colored arrows are the three merge layers from the worked example, using the same colors as the equations above.

# 24 Rank Based Sort

An excerpt from Frederico Mazzone et al. [7] discusses a solution that applies matrix computation to reduce the comparison depth many sorting methods struggle with. Comparison, swap-based techniques, and arg-max functions that normal sorting methods use are computationally expensive in FHE. We would prefer to use these methods minimally.

With CKKS's addition, multiplication, and rotation, we can evaluate a sorting method without swapping to elements.

### 24.0.1 Methods

To understand the process of the rank-based sorting, we will need the following methods. The methods will be followed with either L or R to signify left and right:

- CMP (Comparison)
- IND (Indicator)
- SUM
- REPL (Replicate)
- RANK
- EQ (Equality)

### 24.0.2 Assumption

Before conducting the sort, we need to take care of one main assumption.

- All elements in the vector are distinct, unless we consider tie corrections

### 24.0.3 Rank

First, we need to find the overall order statistics of each element. To find which element is in the "k-th" place, we need to start with the rank function. The rank will allow us to determine which element should be in the i'th position for our sort.

To determine the rank of a vector:

1. Perform $REPL(v) = A$ to replicate the row vector of n elements into a matrix of n x n.
2. $CMP(A, A^T)$ to compare the replicated matrix with its transpose
   - To do so, we will take the difference between the two matrices to output a matrix of -1, 0, 1 values: $A - A^T = M$
3. Let $M = sign(M)$ isolating the positive values to create a matrix of just 0 and 1
4. $R = SUM(M)$ by summing either the columns or rows depending on the initial difference matrix, we will have a final output of the statistic rank from 0 to n-1
5. $R = R + 1$ by adding 1 to each element in R, we will end with a vector of ranks 1 to n

The example below works through $x = [30, 10, 40, 20]$, following the value 30 in orange.

$$\underset{A}{\begin{bmatrix} \mathbf{30} & 10 & 40 & 20 \\ \mathbf{30} & 10 & 40 & 20 \\ \mathbf{30} & 10 & 40 & 20 \\ \mathbf{30} & 10 & 40 & 20 \end{bmatrix}} - \underset{A^T}{\begin{bmatrix} \mathbf{30} & \mathbf{30} & \mathbf{30} & \mathbf{30} \\ 10 & 10 & 10 & 10 \\ 40 & 40 & 40 & 40 \\ 20 & 20 & 20 & 20 \end{bmatrix}} = \underset{M}{\begin{bmatrix} \mathbf{0} & -20 & 10 & -10 \\ \mathbf{20} & 0 & 30 & 10 \\ \mathbf{-10} & -30 & 0 & -20 \\ \mathbf{10} & -10 & 20 & 0 \end{bmatrix}}$$

**Step 1.** Replicate $x$ into the rows of $A$, so $A_{ij} = x_j$ and $A^T_{ij} = x_i$. Subtracting gives $M_{ij} = x_j - x_i$.

$$\underset{\text{sign}(M)}{\begin{bmatrix} \mathbf{0} & 0 & 1 & 0 \\ \mathbf{1} & 0 & 1 & 1 \\ \mathbf{0} & 0 & 0 & 0 \\ \mathbf{1} & 0 & 1 & 0 \end{bmatrix}} \xrightarrow{\text{SUM}} \underset{R}{\begin{bmatrix} \mathbf{2} \\ 0 \\ 3 \\ 1 \end{bmatrix}} \xrightarrow{+1} \underset{R+1}{\begin{bmatrix} \mathbf{3} \\ 1 \\ 4 \\ 2 \end{bmatrix}}$$

**Step 2.** Keep only the positive entries, so the entry is 1 if $M_{ij} > 0$ and 0 otherwise. Column $j$ now marks the values smaller than $x_j$, so summing each column gives ranks from 0 to $n-1$. Adding 1 gives ranks from 1 to $n$, and 30 has rank 3.

### 24.0.4 Rank-Based Sort

To sort a vector of degree n, we need the following:

- A matrix of replicated rows of our input (Matrix X)
- A matrix of replicated rank row vectors (Matrix A)
- A plain text constant matrix with all elements in each row being one singular number from 1 to n (Matrix B)

The process goes as follows:

1. $C = IND(A - B)$ to set all elements that are 0 to value 1 with rest being 0
2. Take the product of our replicated matrix and the indicator matrix $P = C * X$
3. Sum the rows $f = SUM(P)$
4. By transposing our final matrix, we will then end with a sorted row vector $f$

The example below continues $x = [30, 10, 40, 20]$ with the ranks $[3, 1, 4, 2]$ found above, following the value 30 in orange. Here the matrix of replicated ranks plays the role of $A$ in step 1, $B$ has row $i$ filled with $i$, and the replicated input is the $A$ of step 2, written $X$ to keep them apart.

$$\underset{A}{\begin{bmatrix} \mathbf{3} & 1 & 4 & 2 \\ \mathbf{3} & 1 & 4 & 2 \\ \mathbf{3} & 1 & 4 & 2 \\ \mathbf{3} & 1 & 4 & 2 \end{bmatrix}} - \underset{B}{\begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \\ 4 & 4 & 4 & 4 \end{bmatrix}} \xrightarrow{\text{IND}} \underset{C}{\begin{bmatrix} \mathbf{0} & 1 & 0 & 0 \\ \mathbf{0} & 0 & 0 & 1 \\ \mathbf{1} & 0 & 0 & 0 \\ \mathbf{0} & 0 & 1 & 0 \end{bmatrix}}$$

**Step 1.** Subtracting $B$ from $A$ gives 0 exactly where the rank in column $j$ equals the row number $i$, and IND turns those zeros into 1s. Each row of $C$ marks the one input whose rank is that row number, and 30 (rank 3) is marked in row 3.

$$\underset{C}{\begin{bmatrix} \mathbf{0} & 1 & 0 & 0 \\ \mathbf{0} & 0 & 0 & 1 \\ \mathbf{1} & 0 & 0 & 0 \\ \mathbf{0} & 0 & 1 & 0 \end{bmatrix}} * \underset{X}{\begin{bmatrix} \mathbf{30} & 10 & 40 & 20 \\ \mathbf{30} & 10 & 40 & 20 \\ \mathbf{30} & 10 & 40 & 20 \\ \mathbf{30} & 10 & 40 & 20 \end{bmatrix}} = \underset{P}{\begin{bmatrix} \mathbf{0} & 10 & 0 & 0 \\ \mathbf{0} & 0 & 0 & 20 \\ \mathbf{30} & 0 & 0 & 0 \\ \mathbf{0} & 0 & 40 & 0 \end{bmatrix}} \xrightarrow{\text{SUM}} \underset{f}{\begin{bmatrix} 10 \\ 20 \\ \mathbf{30} \\ 40 \end{bmatrix}}$$

**Step 2.** The entrywise product $P = C * X$ keeps one input per row. Summing each row of $P$ gives $f = [10, 20, 30, 40]$, and transposing it gives the sorted row vector, with 30 landing in position 3, its rank.

### 24.0.5 Ties

When a given vector includes non distinct elements, we will have to perform a tie correction.

To do so, we will need the following:

- A matrix of replicated rows of our input: A
- An upper triangular matrix U

To do so:

1. We will first create an equality matrix by comparing the replicated matrix and it's transpose outputting 1 if the index is equal and 0 else.

   $M1 = EQ(A, A^T)$

2. Multiply M1 with a upper triangular matrix to convert the tie elements into two distinct elements

   $M2 = M1 * U$

3. $s1 = SUM(M1)$ and $s2 = SUM(M2)$
4. $O = s1 - s2$
   - This final vector will have offset values of only 1 and 0.
5. $t = R + O$ where $t$ will be our final rank vector by conducting the sum of the original ranks and the offset

The example below works through $x = [30, 10, 10, 20]$, with the entries belonging to the tied values in orange.

$$\underset{A}{\begin{bmatrix} 30 & \mathbf{10} & \mathbf{10} & 20 \\ 30 & \mathbf{10} & \mathbf{10} & 20 \\ 30 & \mathbf{10} & \mathbf{10} & 20 \\ 30 & \mathbf{10} & \mathbf{10} & 20 \end{bmatrix}} == \underset{A^T}{\begin{bmatrix} 30 & 30 & 30 & 30 \\ \mathbf{10} & \mathbf{10} & \mathbf{10} & \mathbf{10} \\ \mathbf{10} & \mathbf{10} & \mathbf{10} & \mathbf{10} \\ 20 & 20 & 20 & 20 \end{bmatrix}} = \underset{M_1}{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \mathbf{1} & \mathbf{1} & 0 \\ 0 & \mathbf{1} & \mathbf{1} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}$$

**Step 1.** Replicate $x$ into the rows of $A$, so $A_{ij} = x_j$ and $A^T_{ij} = x_i$. Then compare the two entry by entry: $(M_1)_{ij} = 1$ if $A_{ij} = A^T_{ij}$, and 0 otherwise.

$$\underset{M_1}{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \mathbf{1} & \mathbf{1} & 0 \\ 0 & \mathbf{1} & \mathbf{1} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}} \odot \underset{U}{\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & \mathbf{1} & \mathbf{1} & 1 \\ 0 & \mathbf{0} & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}} = \underset{M_2}{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \mathbf{1} & \mathbf{1} & 0 \\ 0 & \mathbf{0} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}$$

**Step 2.** Multiply $M_1$ entry by entry with an upper triangular matrix of ones, where $U_{ij} = 1$ if $i \leq j$ and 0 otherwise. This gives $(M_2)_{ij} = (M_1)_{ij} U_{ij}$, which drops the matches below the diagonal.

$$\overset{s_1}{\begin{bmatrix}1\\ \mathbf{2}\\ \mathbf{2}\\ 1\end{bmatrix}} - \overset{s_2}{\begin{bmatrix}1\\ \mathbf{2}\\ \mathbf{1}\\ 1\end{bmatrix}} = \overset{O}{\begin{bmatrix}0\\ \mathbf{0}\\ \mathbf{1}\\ 0\end{bmatrix}} \qquad \overset{R}{\begin{bmatrix}4\\ \mathbf{1}\\ \mathbf{1}\\ 3\end{bmatrix}} + \overset{O}{\begin{bmatrix}0\\ \mathbf{0}\\ \mathbf{1}\\ 0\end{bmatrix}} = \overset{t}{\begin{bmatrix}4\\ \mathbf{1}\\ \mathbf{2}\\ 3\end{bmatrix}}$$

**Step 3.** Sum the rows of each matrix, so $(s_1)_i = \sum_j (M_1)_{ij}$ and $(s_2)_i = \sum_j (M_2)_{ij}$. Subtract them to get the offset $O$, then add $O$ to the ranks $R$ so tied values get different ranks in $t$.

# 25 Max Function

As a consequence of (any) sorting we get the max function for free, since a single $n-1$ bubble pass has allowed the largest element to "float" to the rightmost element. A reversed setup reveals the minimum value if desired. An $n$-length mask vector $(0, 0, ..., 0, 1)$ can be used to isolate the maximum if needed. A dot product or intravector sum can then be used to collapse the resulting vector into the maximum element as a single scalar.

However, if we *just* want the max function, we can do quite a bit better and get it in $O(\log(n))$ time. The idea is to separate the vector into $n/4$ bubbles, and on each turn act in a way that moves the max into the "better half" of the remaining "contenders for the max", like a tournament. This is similar to using our bubbles $(i, j)$ encoded by $c_i$ and $c_j$, but instead of using 1-hot vectors, we will generalize $c_*$ to be 0-1 vectors to do more comparisons at the same time.

Let $x = (x_1, ..., x_{n/2})$. We iterate on $i$, starting with $i = 0$.

1. At the start of iteration $i$, construct constant binary vectors $c_L$ and $c_R$ which have a 1 at every index $k$ that satisfies $k = 2^i - 1 \pmod{2^{i+1}}$ for $c_L$, and $k = 2^{i+1} - 1 \pmod{2^{i+1}}$ for $c_R$,

    - On the first iteration $i = 0$, we have $c_L = (1, 0, 1, 0, \cdots)$ and $c_R = (0, 1, 0, 1, \cdots)$.
    - On subsequent iterations the 1s are separated by double the distance per iteration. For example, for $i = 2$, the period will be 8: $c_L = (0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, \cdots)$ and $c_R = (0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, \cdots)$ (indexed from 0).

    - The $c_L$ and $c_R$ will match up pairs of numbers, acting as a kind of simultaneous bubble.

2. We now match up the bubbles by splitting the relevant entries into two vectors holding the two halves of the bubbles, and cycling the left half so it matches the right:

    - Construct vectors $L' = x * c_L$ and $R = x * c_R$.
    - On iteration $i$, let $L := \texttt{cyc}_{2^i}(L')$, where $\texttt{cyc}_m$ moves every entry $m$ places to the right.
    - Compute $c = L - R$. This will give a vector populated by zeros, and $L_i - R_i$ in populated index positions $i$.
    - In $c$, any positive values in c represent index positions where the $L$ value is larger, and vice-versa for negative values and $R$.

3. We now create a vector with only the "winners" in their respective positions.
    - Compute $e = L * \texttt{sign}(c)$. Here, as in PairSort, `sign` returns 1 when its input is nonnegative and 0 otherwise, so ties go to $L$ and, since tied values are equal, nothing is lost. This contains only $L$ values that were greater in their respective bubble comparisons.
    - Similarly, computing $f = R * (1 - \texttt{sign}(c))$ will return a vector of only 0's and $R$ values that were greater in their respective comparisons.
    - $x' := e + f$ gives a vector with only the winners from each respective bubble.
4. Treating $x'$ as the new $x$, after $\log_2(n/2)$ iterations $x$ will only have the max of the original $x$, in the rightmost position.
    - We note that after iteration $i-1$ and before iteration $i$, the new $x$ will have nonzero entries only in places (indexed starting at 0) whose binary expansions have $i$ 1s at the right. For example, after the first iteration only the odd indices will have nonzero entries; the other "loser" entries have been eliminated.

Figure 25.1 shows the vector after each round for eight slots, $x = (5, 2, 8, 1, 9, 3, 7, 4)$, numbered from 0. In round $i$ the left slot of each pair is rotated onto its right partner and compared, and the larger value is kept in the right slot, so the number of live slots halves each round. Round 1 is shown in full, round 2 is elided, and after $\log_2(8) = 3$ rounds only slot 7 is left, holding the max. The brackets mark the pairs compared next.

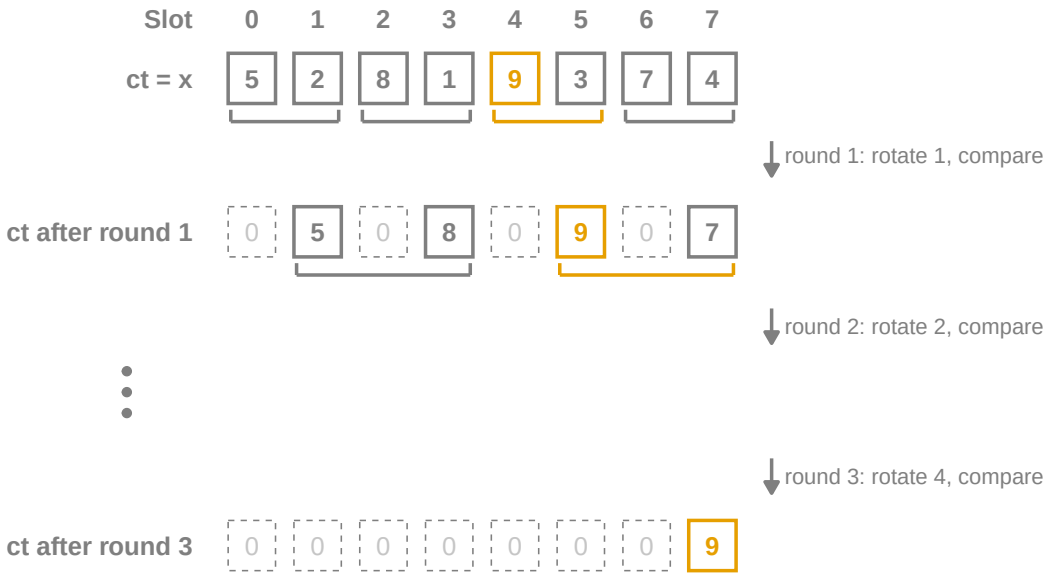



Figure 25.1: The max of eight slots, computed round by round, ends with 9 alone in slot 7.

**Remark**

An immediate "improvement" is that we do not actually need to make these different $C_L$ and $C_R$;

1. We can keep $c_L = (1, 0, 1, 0, \cdots)$ and $c_R = (0, 1, 0, 1, \cdots)$ the whole time;
2. If we do this, then we will still get the same comparisons that lead us to the max in this algorithm, however, the other 1's will be doing something with the "eliminated" entries, which we can ignore.

Thus, overall we can be more efficient by doing less, though we kept our exposition since we think it does a better job of focusing on the key comparisons.

# Part VII

# Discrete Computational Models

In this section, we change the computational model. So far, each slot of a ciphertext has held an arbitrary complex number, and functions such as sign could only be approximated (recall the disclaimer in Discrete Math). Here, we instead assume our inputs come from a finite, discrete set, such as integers in a fixed range. The allowed moves from Setup stay the same; what changes is the kind of input we assume and how we store it in the slots.

In the case that our domain has finite discrete values, any function on that domain can be represented by a polynomial. Since there are a finite number of inputs, we can interpolate a polynomial that agrees with the function at every one of them.

For example, the indicator of $x > 0$, which we could only approximate with series earlier, can be matched *exactly* by a low-degree polynomial when $x$ only takes a few integer values; we will see this in Extended Binary Numbers. CKKS arithmetic is still approximate, so the computed values will be close to 0 or 1 rather than exactly equal, with errors on the order of the precision from Setup.

**Remark**

Of course, this depends on the user supplying their data in a discrete form. Since users encrypt their own inputs, they can supply not only $x$ but also its binary digits or its root-of-unity form (much like supplying inverses in Division). The catch is that not every operation preserves this form: adding binary digits requires carrying, and multiplying numbers stored as roots of unity calls for a lookup table. So each choice of form makes certain things easier and other things harder.

A finite domain may be encoded by integers modulo some fixed $n$. We explore two such encodings: roots of unity and $k$-bit binary. With this in mind, we discuss uses such as modular arithmetic, the sign function, and sorting, and consider the computational costs and numerical accuracy of these methods.

# 26 Roots of Unity

One way to store discrete data and bounded data is as *roots of unity*. Let $\zeta$ be a primitive $N$-th root of unity for some fixed prime $N$. Then we represent $a$ by $\zeta^a$ for $a \in \{0, 1, \ldots, N-1\}$. Here, $\zeta = e^{\frac{2\pi i}{N}}$.

Then we can do **modular addition** mod $N$ via multiplication:

$$(\zeta^k)(\zeta^l) = \zeta^{k+l}.$$

Figure 26.1 shows this for $N = 7$. Each $\zeta^a$ is a point on the unit circle, and multiplying two points adds their angles, so going past $\zeta^0$ wraps the exponent around mod $N$. Here, $\zeta^4 \cdot \zeta^5 = \zeta^9 = \zeta^2$.

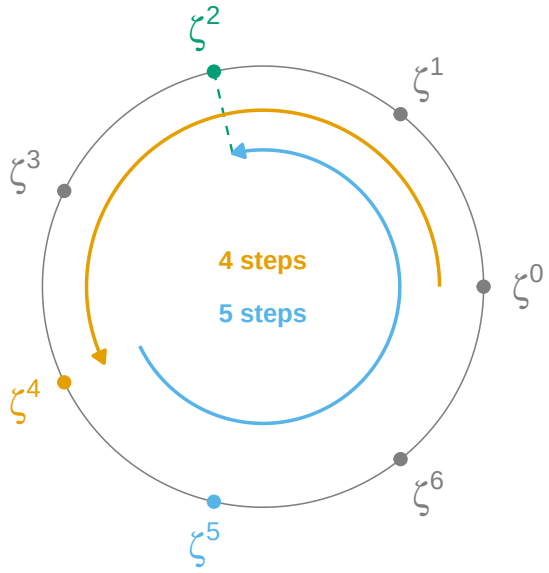



Figure 26.1: Modular addition on the unit circle for $N = 7$. Rotating by 4 steps (orange) and then 5 steps (blue) lands on $\zeta^2$ (green), since $4 + 5 \equiv 2 \pmod 7$.

However, **modular multiplication** becomes much more difficult.

An option is to interpolate a bivariate polynomial such that $P(\xi^a, \xi^b) = \xi^{ab}$. Since $n$ is finite, the Cayley table of $\mathbb{Z}/n\mathbb{Z}$ is also finite. Thus, we should be able to interpolate a polynomial which functions as a lookup table

# 27 Lookup Tables

Suppose that you want to perform an operation $f(a, b)$. Recall that $a, b \in \{0, ..., n-1\}$ and that $a, b$ are stored as roots of unity $\zeta^a, \zeta^b$ where $\zeta = e^{2\pi i/n}$. The desired output may be a value in either root of unity form or as a regular encoded value.

Since there are finitely many possible inputs, we could calculate every possible value of $f(a, b)$ beforehand, without encryption. Then, when we receive encrypted inputs, we would like to use this table to obtain the encrypted answer.

This is the idea behind lookup tables. In our applications, they are relevant to modular multiplication, inverses when they exist, sign functions, and more.

One approach is to interpolate a polynomial that gives the desired output at every possible input. Evaluating this polynomial on the encrypted inputs then performs the lookup. The table and polynomial coefficients can be public while the inputs and resulting output remain encrypted.

Chung, Kim, Kim, and Lee [8] use this approach with inputs encoded as roots of unity. Suppose there are $n$ possible values. For a lookup table $T$ with one input, we construct a polynomial satisfying

$$f(\zeta^a) = \zeta^{T(a)}.$$

There are $n$ interpolation points, so the polynomial has at most degree $n-1$.

For operations with multiple inputs, we will consider two approaches. First, we will consider the approach which interpolates a polynomial in multiple variables while keeping inputs separate. Then, we will consider the approach which combines inputs into a single value and interpolates a single variable polynomial.

### 27.0.1 Multivariable Interpolation

Chung, Kim, Kim, and Lee [8] use multivariable interpolation. For example, modular multiplication requires a polynomial satisfying

$$f(\zeta^a, \zeta^b) = \zeta^{ab}.$$

Instead of combining two inputs into a single value, Chung, Kim, Kim, and Lee construct a polynomial in two variables. Suppose each input is an $n$-th root of unity. Let $d_{ab}$ be the desired encoded output for the input pair $(\zeta^a, \zeta^b)$. We want a polynomial satisfying

$$f(\zeta^a, \zeta^b) = d_{ab}$$

for every possible pair of inputs.

### 27.0.2 Example: $n = 4$

Each input is a fourth root of unity.

The lookup table has sixteen entries:

| | $v = 1$ | $v = \zeta$ | $v = \zeta^2$ | $v = \zeta^3$ |
|---|---|---|---|---|
| $u = 1$ | $d_{00}$ | $d_{01}$ | $d_{02}$ | $d_{03}$ |
| $u = \zeta$ | $d_{10}$ | $d_{11}$ | $d_{12}$ | $d_{13}$ |
| $u = \zeta^2$ | $d_{20}$ | $d_{21}$ | $d_{22}$ | $d_{23}$ |
| $u = \zeta^3$ | $d_{30}$ | $d_{31}$ | $d_{32}$ | $d_{33}$ |

First, hold the second input fixed at $v = \zeta^b$. We can interpolate a polynomial $p_b(u)$ satisfying

$$p_b(\zeta^a) = d_{ab}, \qquad a = 0, 1, 2, 3.$$

Each $p_b$ has degree at most three. Doing this for every column gives four polynomials.

$$p_0(u), \quad p_1(u), \quad p_2(u), \quad p_3(u).$$

Next, we need to select the correct column using $v$. Define the Lagrange polynomials

$$L_b(v) = \prod_{\substack{0 \le k \le 3 \\ k \neq b}} \frac{v - \zeta^k}{\zeta^b - \zeta^k}.$$

These satisfy

$$L_b(\zeta^k) = \begin{cases} 1, & k = b, \\ 0, & k \neq b. \end{cases}$$

For example,

$$L_0(v) = \frac{(v - \zeta)(v - \zeta^2)(v - \zeta^3)}{(1 - \zeta)(1 - \zeta^2)(1 - \zeta^3)}.$$

We can now combine the column polynomials:

$$f(u,v) = p_0(u)L_0(v) + p_1(u)L_1(v) + p_2(u)L_2(v) + p_3(u)L_3(v).$$

When $v = \zeta^b$, all terms go to 0 except $p_b(u)$. Thus,

$$f(\zeta^a, \zeta^b) = p_b(\zeta^a) = d_{ab},$$

as desired.

The column polynomials themselves can be written as

$$p_b(u) = \sum_{a=0}^{3} d_{ab} L_a(u).$$

Substituting gives

$$\boxed{f(u,v) = \sum_{a=0}^{3}\sum_{b=0}^{3} d_{ab} L_a(u) L_b(v).}$$

Each Lagrange polynomial has degree three, so $f$ has degree at most three in each variable. After expanding, it has the form

$$f(u,v) = \sum_{i=0}^{3}\sum_{j=0}^{3} A_{ij} u^i v^j.$$

There are sixteen coefficients, but the total degree is at most six, coming from the term $u^3v^3$.

### 27.0.3 Combining Inputs

Alternatively, we could encode a pair of elements as a single value, and then interpolate a polynomial in one variable. We can do all of this publicly beforehand, so only evaluating the polynomial needs to happen in an encrypted setting.

We might consider something like

$$t(w,z) = nw + z.$$

This distinguishes every ordered pair of $n$th roots of unity, and gives $n^2$ points to interpolate on. Thus, the lookup table may be represented as polynomial of degree at most $n^2 - 1$. Although the polynomial is higher degree, this method has a particular advantage with symmetric functions.

### 27.0.4 Exploiting Symmetry

Consider the case of modular multiplication. This operation is symmetric. That is, $f(a, b) = f(b, a)$. It seems reasonable that with symmetry, we should be able to simplify our lookup table in some way.

| | $b = 0$ | $b = 1$ | $b = 2$ | $b = 3$ |
|---|---|---|---|---|
| $a = 0$ | $d_{00}$ | $d_{01}$ | $d_{02}$ | $d_{03}$ |
| $a = 1$ | $d_{10}$ | $d_{11}$ | $d_{12}$ | $d_{13}$ |
| $a = 2$ | $d_{20}$ | $d_{21}$ | $d_{22}$ | $d_{23}$ |
| $a = 3$ | $d_{30}$ | $d_{31}$ | $d_{32}$ | $d_{33}$ |

Figure 27.1: A symmetric lookup table for $n = 4$.

The current construction does not use the symmetry of the table, because $t(w, z) \neq t(z, w)$ when $w \neq z$. Ideally, they would be equal so that we can use fewer interpolation points and get a lower degree.

So, we want to create a function $t(w, z)$ such that $t(w, z) = t(u, v)$ if and only if

$$(u, v) = (w, z) \quad \text{or} \quad (u, v) = (z, w).$$

Instead, consider something with a nonlinear term:

$$t(w, z) = w + z + cwz.$$

This is symmetric, but we need to choose $c$ so that two different unordered pairs do not give the same value.

If two pairs give the same value, then

$$w + z + cwz = u + v + cuv,$$

so

$$c(wz - uv) = u + v - w - z.$$

If $wz = uv$, then their sums must also be equal. Having the same sum and product means they are the same unordered pair.

Otherwise, there is just one value of $c$ that would make these two pairs collide:

$$c = \frac{u + v - w - z}{wz - uv}.$$

Since there are only finitely many pairs, there are only finitely many values of $c$ to avoid. So we can choose a $c$ that works.

Then, we interpolate a polynomial $q$ such that

$$q(t(\zeta^a, \zeta^b)) = \zeta^{ab}.$$

Now we only need one interpolation point for each unordered pair. This gives us $n(n+1)/2$ points instead of $n^2$, and therefore a degree bound of $n(n+1)/2 - 1$ instead of $n^2 - 1$.

For example, let $n = 3$ and let $\zeta = e^{2\pi i/3}$. Then

$$\zeta^3 = 1$$

In this case, we can choose $c = 1$, giving

$$t(w, z) = w + z + wz.$$

We only need to consider the following six unordered pairs:

| $(w, z)$ | $t(w, z)$ | Desired output $\zeta^{ab}$ |
|---|---|---|
| $(1, 1)$ | $3$ | $1$ |
| $(1, \xi)$ | $1 + 2\zeta$ | $1$ |
| $(1, \xi^2)$ | $1 + 2\zeta^2$ | $1$ |
| $(\xi, \xi)$ | $2\zeta + \zeta^2$ | $\zeta$ |
| $(\xi, \xi^2)$ | $\zeta^2 + \zeta + 1$ | $\zeta^2$ |
| $(\xi^2, \xi^2)$ | $2\zeta^2 + \zeta$ | $\zeta$ |
| $(\xi^2, 1)$ | $1 + 2\zeta^2$ | $1$ |
| $(\xi^2, \xi)$ | $\zeta^2 + \zeta + 1$ | $\zeta^2$ |
| $(\xi^2, \xi^2)$ | $2\zeta^2 + \zeta$ | $\zeta$ |

There are six values in the middle column which are distinct, so we can interpolate a polynomial $q$ of degree at most 5 with these inputs and outputs. We also notice that values which aren't distinct have corresponding flipped $w$ and $z$ values.

Explicitly, label the interpolation points

$$(t_0, t_1, t_2, t_3, t_4, t_5) = (3, 1 + 2\zeta, 1 + 2\zeta^2, \zeta - 1, 0, \zeta^2 - 1).$$

Let

$$L_j(T) = \prod_{\substack{0 \le k \le 5 \\ k \ne j}} \frac{T - t_k}{t_j - t_k}.$$

Then our polynomial is

$$q(T) = L_0(T) + L_1(T) + L_2(T) + \zeta L_3(T) + \zeta^2 L_4(T) + \zeta L_5(T).$$

Thus, for any $a, b \in \{0, 1, 2\}$,

$$q\left(\zeta^a + \zeta^b + \zeta^{a+b}\right) = \zeta^{ab}.$$

We can construct $q$ publicly beforehand. Given encrypted inputs $w$ and $z$, we would then evaluate $q(w + z + wz)$.

### 27.0.5 Exploiting Symmetry with Multivariate Interpolation

We can also use symmetry to our advantage while keeping variables separate. Since our polynomial represents a symmetric function, $(P(u, v) = P(v, u))$. In the expanded form,

$$P(u, v) = \sum_{i,j=0}^{n-1} A_{ij} u^i v^j,$$

we therefore have $(A_{ij} = A_{ji})$. This lets us group terms with matching coefficients as follows.

$$P(u, v) = \sum_{i=0}^{n-1} A_{ii} (uv)^i + \sum_{0 \le i < j \le n-1} A_{ij} (uv)^i \left(u^{j-i} + v^{j-i}\right).$$

We can compute the powers of $(uv)$ and the sums $(u^k + v^k)$ instead of individually multiplying every term $u^i v^j$

To compute the $(u^k + v^k)$, we can use factorization. For example, since $i^2 = -1$,

$$u^2 + v^2 = (u + iv)(u - iv).$$

This requires only one multiplication, rather than the two needed to square $u$ and $v$ separately.

We would like to generalize this for $k > 2$. We know that we can factor any polynomial using its complex roots.

First, consider the single variate case:

$$x^k + 1 = \prod_{i=1}^{k}(x - r_i).$$

Where $r_i$ is the $i$-th $k$-th root of $-1$ .

When $v = 0$ our problem is trivial. For $v \neq 0$, substitute $x = u/v$ to get

$$\left(\frac{u}{v}\right)^k + 1 = \prod_{i=1}^{k}\left(\frac{u}{v} - r_i\right).$$

Multiplying by $v^k$ and distributing gives

$$u^k + v^k = \prod_{r=1}^{k}(u - r_i v).$$

Evaluating this requires $k - 1$ multiplications. This can be more efficient for small $k$, but is not efficient for larger $k$.

Alternatively, we can use the identity

$$u^{2k} + v^{2k} = (u^k + v^k)^2 - 2(uv)^k.$$

If $(uv)^k$ and \$ u$^{k+v}$k\$ are already available, this requires just one ciphertext-ciphertext multiplication.

### 27.0.6 Example: Construction and Contrasting of Modular Multiplication for n = 4

Below are the two methods of polynomial interpolation on the multiplication table with $n = 4$.

Bivariate interpolation gives:

$$\begin{aligned} P(u,v) = \frac{1}{4}[1 + v + v^2 + v^3 \\ &+ u(1 - iv - v^2 + iv^3) \\ &+ u^2(1 - v + v^2 - v^3) \\ &+ u^3(1 + iv - v^2 - iv^3)]. \end{aligned}$$

For the bivariate method, calculating $u^2, u^3, v^2, v^3$ takes four multiplications. There are also nine mixed terms $u^i v^j$ with $i, j \in \{1, 2, 3\}$, giving thirteen multiplications altogether.

This method needs only three layers: two to obtain powers through $u^3$ and $v^3$, and one more for the mixed terms.

With symmetry grouping, we have

$$\begin{aligned} P(u,v) = \frac{1}{4}[&1 + (u+v) + (u^2+v^2) + (u^3+v^3) - iuv \\ &- uv(u+v) + iuv(u^2+v^2) + (uv)^2 \\ &- (uv)^2(u+v) - i(uv)^3]. \end{aligned}$$

We can evaluate this in three multiplication layers:

| Layer | Values computed | Multiplications |
|---|---|---|
| 1 | $u^2,\ v^2,\ uv$ | 3 |
| 2 | $u^3 = u^2u,\ v^3 = v^2v,\ (uv)^2,\ uv(u+v),\ uv(u^2+v^2)$ | 5 |
| 3 | $(uv)^2(u+v),\ (uv)^3 = (uv)^2(uv)$ | 2 |

Thus, this evaluation uses $3 + 5 + 2 = 10$ multiplications and has multiplicative depth 3.

For the single-variable polynomial, let $c = 2$. Thus,

$$t(u,v) = u + v + 2uv.$$

$$\begin{aligned} Q(t) = 1 + \frac{1}{7072000}[&(12507520 - 191360i)t \\ &+ (3844256 - 421408i)t^2 \\ &- (2688960 + 341120i)t^3 \\ &- (1056912 + 124384i)t^4 \\ &- (173756 + 8892i)t^5 \\ &- (47656 + 2392i)t^6 \\ &- (24280 + 1560i)t^7 \\ &+ (4092 + 1144i)t^8 \\ &+ (3181 + 117i)t^9]. \end{aligned}$$

For the combined-input method, calculating $t = u + v + cuv$ takes one multiplication. Then, calculating $t^2, \ldots, t^9$ takes eight more. This method has multiplicative depth 5: One layer to calculate $t$, and four more to get all powers through $t^9$.

Overall, the bivariate method has lower multiplicative depth but a greater number of multiplications.

| | Univariate | Bivariate | Bivariate with symmetry |
|---|---|---|---|
| Encrypted multiplications | 9 | 13 | 10 |
| Multiplicative depth | 5 | 3 | 3 |

# 28 A Binary Approach to Inequality and Sorting

In this computational model, each ciphertext vector with length $n/2$ stores $n/(2k)$ numbers, using $k$ components to store each number. The user supplies not only (or instead of, but it seems fairly cheap to have both representations) $x$, but also the digits of $x$ (assuming integers for simplicity, maybe $x = (x_{k-1} \cdots x_0)|_b$ in base $b$, and each $x_i$ represents an element of $\{0, 1, \ldots, b-1\}$.

For this example, assume $b = 2$ so things are stored in binary. We will show that the inequality function can be done "exactly" as polynomials instead of approximated with a Taylor expansion.

1. When $x$ and $y$ are *single bits*, the equality function $(x == y)$ is an algebraic condition

$$(x == y) := \frac{1}{2}(xy + (1-x)(1-y)),$$

2. and whether $(x > y)$ is also an exact condition, namely

$$(x > y) := (x == 1) * (y == 0);$$

3. Therefore, when $x$ and $y$ are $k$-bit numbers (assuming positive), we can write $x > y$ as an OR (and thus a polynomial) of the following values:

   - $(x_{k-1} > y_{k-1})$ OR
   - $((x_{k-1} == y_{k-1})$ AND $(x_{k-2} > y_{k-2}))$ OR
   - $((x_{k-1} == y_{k-1})$ AND $(x_{k-2} == y_{k-2})$ AND $(x_{k-3} > y_{k-3}))$ OR … This is simply *some* polynomial in the $x_i$ and $y_i$, so it is obtainable under FHE.

Thus, doing inequalities with high accuracy at least becomes possible when we do binary operations. The (high!) tradeoff is that adding and multiplying become difficult (due to carries). But if we are only doing comparative work, then much of the sorting / maxing related operations in Section IV becomes very viable.

# 29 Comparison and Extended Binary Numbers

In this section, we show we can compare an input integer ciphertext $c$ with 0, meaning a function which outputs 1 if $c > 0$ and 0 otherwise. (it will be clear how to do variants of this, such as with nonstrict inequalities). Our main trick is to use what we call the **extended binary expansion** of $c$.

Define $S_1 := -1, 0, 1$. We say that a number $a$ is in **extended binary** form if it is represented as

$$a = a_0 + a_1 2 + \cdots + a_{b-1} 2^{b-1},$$

where each digit satisfies $a_i \in S_1$.

**Remark ($S_1$):**

The main intuition is this: we need the slightly larger digit set $S_1$ because the difference of two ordinary binary numbers has digits in $\{-1, 0, 1\}$. This avoids needing to perform carrying, which is hard in CKKS.

Now:

1. Assume that we have the extended binary digits of $c$ as $c_0, c_1, \ldots, c_{b-1}$, in little-endian order, where each $c_i \in S_1$.
2. We construct the functions "$\geq 0$" and "$= 0$" on the finite set $S_1$. Since $S_1$ is finite, these functions are polynomials on $S_1$! (furthermore, with operations that are not likely to accumulate a lot of floating point error)
   - For example, on $S_1$, the function $\mathbf{1}_{x>0}$ is given by
   $$\frac{x^2 + x}{2}.$$
   Indeed, this is $0, 0, 1$ at $x = -1, 0, 1$ respectively.
   - Similarly, the function $\mathbf{1}_{x=0}$ is given by $1 - x^2$.
3. This allows us to construct a comparison function on the space of extended binary numbers. The key logic is that an extended binary number
   $$a = a_0 + a_1 2 + \cdots + a_{b-1} 2^{b-1}$$

is positive if and only if at least one of the following events occurs:

$$a_i = 1 \quad \text{and} \quad a_{i'} = 0 \text{ for all } i' > i.$$

In words: the most significant nonzero digit of (a) is positive.

Therefore,

$$\mathbf{1}_{a>0} = \bigvee_{i=0}^{b-1} \left( (\mathbf{1}(a_i = 1) \wedge \bigwedge_{i'>i} \mathbf{1}(a_{i'} = 0) \right).$$

To see the formula decide a comparison, take $x = 1011_2 = 11$ and $y = 0110_2 = 6$. The orange digit is the most significant nonzero digit of the difference, which is the one that decides the sign.

| | $i=3$ | $i=2$ | $i=1$ | $i=0$ | value |
|---|---|---|---|---|---|
| $x$ | 1 | 0 | 1 | 1 | 11 |
| $y$ | 0 | 1 | 1 | 0 | 6 |
| $x-y$ | **1** | $-1$ | 0 | 1 | 5 |
| $\mathbf{1}(d_i=1)\prod_{i'>i}\mathbf{1}(d_{i'}=0)$ | **1** | 0 | 0 | 0 | $\mathbf{1}_{x>y}=1$ |
| $y-x$ | $-1$ | 1 | 0 | $-1$ | $-5$ |
| $\mathbf{1}(d_i=1)\prod_{i'>i}\mathbf{1}(d_{i'}=0)$ | 0 | 0 | 0 | 0 | $\mathbf{1}_{y>x}=0$ |

**Step 1.** The digits of $x - y$ are $d_i = x_i - y_i$, so $(d_3, d_2, d_1, d_0) = (1, -1, 0, 1)$, and indeed $8 - 4 + 0 + 1 = 5$.

**Step 2.** The term for $i = 3$ is $1 \cdot 1 = 1$, because no higher digit exists, so the OR is 1 no matter what the lower digits do. The lower terms are 0 because each has a nonzero digit above it, so $x > y$.

**Step 3.** Swapping gives digits $(-1, 1, 0, -1)$. The digit $d_2 = 1$ is blocked by $d_3 = -1 \neq 0$, so every term is 0 and $y > x$ is false.

The logical AND and OR operations can themselves be written as polynomials on Boolean values:

$$A \wedge B = AB,$$

and

$$A \vee B = 1 - (1 - A)(1 - B).$$

Thus the comparison function can be implemented as a polynomial in the extended binary digits.

Since the intermediate values are essentially Boolean or integer-valued, this construction avoids the main numerical instability of trying to compare arbitrary real CKKS values directly.

# 30 Index To One-Hot

Using roots-of-unity, we can do a one-hot conversion function (this will be used later for sorting). Specifically, given a ciphertext input for integer $i$, stored in root-of-unity form (from this point we use $\sqrt{-1}$ to distinguish from $i$)

$$\zeta^i = e^{2\pi i\sqrt{-1}/k},$$

where $\zeta = e^{2\pi\sqrt{-1}/k}$, we wish to create a one-hot vector $v$ which is 1 in the $i$-th coordinate and 0 elsewhere. We can do this as follows:

1. Create a vector $v'$ whose $j$-th coordinate is $\zeta^{-j}$.
2. Multiply entrywise by $\zeta^i$: $v'' = v' \cdot \zeta^i$.
3. Then the $j$-th coordinate of $v''$ is $\zeta^{i-j}$. In particular, the $i$-th coordinate is (1), while every other coordinate is a nontrivial $k$-th root of unity.
4. For each entry $w$ of $v''$, compute

   $$1 + w + w^2 + \cdots + w^{k-1}.$$

   This expression equals $k$ if $w = 1$ and equals 0 if $w$ is a nontrivial $k$-th root of unity.
5. Therefore, after dividing by $k$, we obtain the desired vector $v$.

# 31 Discrete sign and notes on sorting

### 31.0.1 Sign function

Suppose we know that our data consists of integers in the range $[-b, b]$, and we want to create a sign function. Notice that modulo $2b+1$, a number in the range $[-b, -1]$ is equivalent to a number in the range $[b+1, 2b]$. Likewise, a positive number sits in the range $[1, b]$.

So, if our inputs are stored as residues modulo $2b+1$, we can create a sign function by interpolating a polynomial which sends $1, \dots, b$ to 1, $b+1, \dots, 2b$ to $-1$, and 0 to 0.

Alternatively, if our values are stored as roots of unity, let $\xi$ be a primitive $(2b+1)$-st root of unity. Then interpolate on the discrete points $\xi^0, \dots, \xi^{2b}$, sending $\xi^1, \dots, \xi^b$ to 1, $\xi^{b+1}, \dots, \xi^{2b}$ to $-1$, and $\xi^0 = 1$ to 0. This gives us the sign of the original integer directly from its root of unity representation.

### 31.0.2 Comparisons

We can use this lookup idea to compare entries without binary expansion. Suppose our integers lie in $[-b, b]$. Their differences lie in $[-2b, 2b]$, so for comparisons we use the larger modulus $n = 4b+1$ and let $\xi$ be a primitive $n$-th root of unity.

Construct a polynomial $S$ by interpolation such that

$$S(\xi^d) = \begin{cases} 0, & d = 0, \\ 1, & 1 \le d \le 2b, \\ -1, & 2b+1 \le d \le 4b. \end{cases}$$

Figure 31.1 shows $S$ for $b = 3$, so $n = 4b+1 = 13$. Each point on the circle is $\xi^d$, which stands for a difference $d$ between two entries, and the label on the point tells you which $d$. The value $S$ gives is the sign of $d$: the orange arc $(d = 1, \dots, 6)$ is the positive differences and gets $+1$, the blue arc $(d = 7, \dots, 12)$ is the negative differences and gets $-1$, and $d = 0$ gets 0. The blue arc holds the negatives because $\xi^d = \xi^{d-13}$, so $d = 7, \dots, 12$ is the same as $d = -6, \dots, -1$.

Given an $m$-length vector $W$, where $W_i = \xi^{a_i}$, we can compare entries $W_i$ and $W_j$ by computing

$$s = S\left(W_i W_j^{n-1}\right).$$

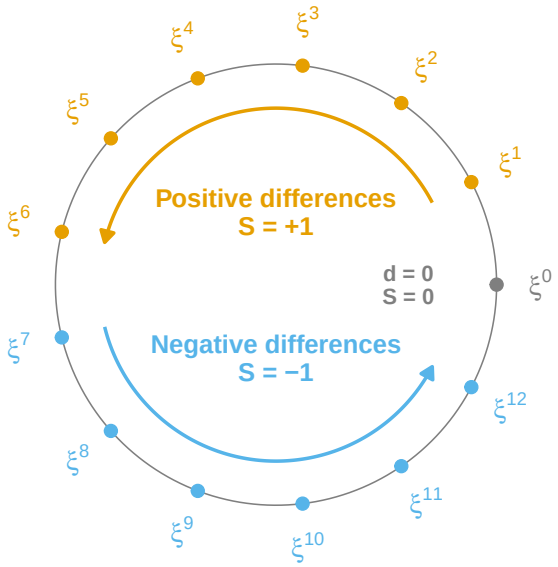



Figure 31.1: The sign polynomial $S$ on the 13th roots of unity ($b = 3$). Each point $\xi^d$ encodes a difference $d$, and $S$ returns its sign. The grey point $\xi^0$ (difference 0) gives 0, the orange arc $\xi^1, \ldots, \xi^6$ (positive differences) gives $+1$, and the blue arc $\xi^7, \ldots, \xi^{12}$ (negative differences $-6, \ldots, -1$) gives $-1$.

Since

$$W_i W_j^{n-1} = \xi^{a_i - a_j},$$

we get $s = 1$ if $a_i > a_j$, $s = -1$ if $a_i < a_j$, and $s = 0$ if they are equal.

If we only want to know whether $a_i > a_j$, we can compute

$$c = \frac{s^2 + s}{2}.$$

However, we can also interpolate this comparison directly. Construct a polynomial $C$ such that

$$C(\xi^d) = \begin{cases} 1, & 1 \le d \le 2b, \\ 0, & d = 0 \text{ or } 2b+1 \le d \le 4b. \end{cases}$$

Then

$$c = C\left(W_i W_j^{n-1}\right)$$

gives the same comparison result without the extra squaring step. This still uses only $n$ interpolation points, since the input is the encoded difference.

### 31.0.3 Bubble sort

We can use bubble sort as a starting point. Bubble sort passes through a given vector, performing repeated pairwise comparisons and swaps.

To sort from smallest to largest, compare neighboring entries $W_i$ and $W_{i+1}$ using

$$c = C\left(W_i W_{i+1}^{n-1}\right).$$

Then $c = 1$ if the entries are out of order and $c = 0$ otherwise.

To swap the entries without branching, define

$$W_i' = (1-c)W_i + cW_{i+1},$$

$$W_{i+1}' = cW_i + (1-c)W_{i+1},$$

leaving every other coordinate unchanged. Both expressions use the original entries. Thus, when $c = 0$, nothing changes; when $c = 1$, the entries swap.

Perform this operation on pairs $(1,2), (2,3), \ldots, (m-1, m)$. These $m-1$ comparisons move the largest integer to the final position, still encoded as a root of unity.

Repeat on the first $m-1$ entries, then the first $m-2$ entries, and so on. After $m(m-1)/2$ comparisons, the vector is sorted by the $a_i$s.

The same PairSort operation can also be used in odd-even and bitonic sorting networks.

### 31.0.4 Sorting by rank

Alternatively, we can use the comparisons to find each entry's final position directly. For distinct inputs, an entry's rank is the number of inputs smaller than it:

$$r_i = \sum_{j=1}^{m} q_{ij}, \qquad q_{ij} = C\left(W_i W_j^{n-1}\right).$$

These comparisons are independent, so they can be computed in parallel.

We can reduce the number of comparisons by comparing only when $i < j$. The diagonal entries are always 0, and, assuming distinct inputs, each entry below the diagonal is 1 minus its reflected entry:

$$q_{ji} = 1 - q_{ij}.$$

Then we count across rows as before. This gives $m(m-1)/2$ comparisons instead of $m^2$.

### 31.0.5 Alternative idea for one-hot vectors

Instead of encoding the ranks as roots of unity, we could keep them as encrypted integers.

For each possible rank $j \in \{0, \ldots, m-1\}$, let $L_j$ be the interpolated polynomial sending $j$ to 1 and every other rank to 0. Then

$$[L_0(r_i), L_1(r_i), \ldots, L_{m-1}(r_i)]$$

is the encrypted one-hot vector for entry $i$.

To get the sorted vector, multiply each one-hot vector by its corresponding encoded entry and add:

$$\sum_{i=1}^{m} W_i [L_0(r_i), L_1(r_i), \ldots, L_{m-1}(r_i)].$$

The output remains encoded as roots of unity.

If we only want the minimum, maximum, or median, we do not need the full one-hot vectors. We can evaluate only the polynomial for the position we need. For example,

$$\sum_{i=1}^{m} W_i L_0(r_i)$$

gives the encoded minimum, while

$$\sum_{i=1}^{m} W_i L_{m-1}(r_i)$$

gives the encoded maximum. For an odd number of entries, the median is at rank $(m-1)/2$.

# 32 Summary

**Summary**

1. A discrete representation of numbers is useful to have as a substitute / in addition to the standard input. Can consider writing down the "digits" as integers or as the roots of unity.

2. With each choice, certain things become much easier (or at least doable) such as the sign / inequality function, but certain things become harder as well (such as addition or multiplication).

3. We give a proof-of-concept in the following repository: https://github.com/fieldextension/discrete-methods

# Part VIII

# Other Explorations

Here, we collect some miscellaneous functions and extensions of previous sections we found interest in during our explorations.

# 33 Dot Products and Matrix Multiplication

Let us work with square matrices for simplicity. A client gives us $A, B \in \mathbb{C}^{n \times n}$ and we want to compute $C = A \cdot B$. For simplicity, we assume that the *rows* of $A$ and the *columns* of $B$ are given. If we only have rows of $B$, we can tweak them to obtain columns.

Recall the dot product of two vectors $x = (x_1, \ldots, x_n)$ and $y = (y_1, \ldots, y_n)$ is defined as $x \cdot y = x_1 y_1 + \cdots + x_n y_n = \sum_{i=1}^{n} x_i y_i$.

1. Suppose we had two encoded vectors $x$ and $y$. We can obtain the vector $x * y = (x_1 y_1, x_2 y_2, \ldots, x_n y_n)$ with element-wise multiplication.
2. We then perform an intravector sum to obtain a vector

$$v(x, y) = \texttt{sum}(x * y) = (d, d, \ldots, d),$$

   where $d = x \cdot y$.
3. We can then e.g. multiply it with a 1-hot mask vector $(0, 0, \ldots, 1, 0, \ldots, 0)$ or use it in any other way we wish.

This directly lets us compute the *entries* of the $n \times n$ matrix $C = A \times B$, since those are dot products:

1. let the encoded first row of $A$ be denoted $A_1 = (a_{11}, a_{12}, \ldots, a_{1n})$ and the encoded first column of $B$ as $B_1 = (b_{11}, b_{21}, \ldots, b_{n1})$.
2. As just stated, we can get $c_{11}$, the $(1, 1)$ entry of $C$, by just taking the dot product $A_1 \cdot B_1$. In general, we can get $c_{ij} = a_i \cdot b_j$, though underneath the hood we are actually getting vectors of the form $v(A_1, B_j) = (c_{ij}, c_{ij}, \ldots, c_{ij})$.

To get actual row / column vectors of $C$, we could get the entries, take some dot products by 1-hot vectors, and add. Specifically, if we wanted the $i$-th row vector $C_i$, we would add

$$(1, 0, \ldots, 0) * v(A_i, B_1) + (0, 1, \ldots, 0) * v(A_i, B_2) + \cdots + (0, \ldots, 0, 1) * v(A_i, B_n).$$

It is possible that we may want to use a different representation of the matrices. For example, let $A$ be arranged such that each row $a_i$ is stored contiguously in ciphertext $A'$:

$$A' = (a_1, a_2, \ldots a_n) = (a_{11}, a_{12}, \ldots, a_{1n}, a_{21}, a_{22}, \ldots, a_{nn})$$

Likewise $B$ with columns $b_j$, denoted

$$B' = (b_{11}, b_{21}, ..., b_{n1}, b_{12}, b_{22}, ..., b_{nn})$$

Using these initial $n^2$ ciphertexts as an example:

$$A' * B' = (a_{11}b_{11}, a_{12}b_{21}, ..., a_{1n}b_{n1}, a_{21}b_{12}, ..., a_{n,n-1}b_{n-1,n}, a_{nn}b_{nn}).$$

Now, we claim that if we can extend summing to do a $k$-th "partial" intravector sum for a $mk$-vector $x$:

$$\begin{aligned}
\texttt{partsum}(x, k) = &(x_1 + \cdots + x_k, \cdots, x_1 + \cdots + x_k, && (33.1)\\
&x_{k+1} + \cdots + x_{2k}, \cdots, x_{k+1} + \cdots + x_{2k}, && (33.2)\\
&\vdots && (33.3)\\
&x_{(m-1)k+1} + \cdots + x_{mk}, \cdots, x_{(m-1)k+1} + \cdots + x_{mk}) && (33.4)
\end{aligned}$$

we would be able to use $\texttt{partsum}(n, A' * B')$ to compute $n$ of the correct terms; for example, $(A \times B)_{1,1}$ in position 1. If we repeat this $n$ times after cycling $A' * B'$ (by $n$ each time), we can combine them to get the final vector. As a concrete example for $n = 2$:

**Concrete** $2 \times 2$ Example:

$$\begin{aligned}
A' &= (a_{11},\ a_{12},\ a_{21},\ a_{22})\\
B' &= (b_{11},\ b_{21},\ b_{12},\ b_{22})\\
B'' &= (b_{12},\ b_{22},\ b_{11},\ b_{21}) = \texttt{cycle}(B',\ n = 2)\\
X &= (1, 0, 0, 1)\\
\overline{X} &= (0, 1, 1, 0)\\
C_x &= A' * B' = (a_{11}b_{11},\ a_{12}b_{21},\ a_{21}b_{12},\ a_{22}b_{22})\\
C_y &= A' * B'' = (a_{11}b_{12},\ a_{12}b_{22},\ a_{21}b_{11},\ a_{22}b_{21})\\
C'_x &= \texttt{partsum}(C_x,\ 2)\\
&= (a_{11}b_{11} + a_{12}b_{21},\ a_{11}b_{11} + a_{12}b_{21},\ a_{21}b_{12} + a_{22}b_{22},\ a_{21}b_{12} + a_{22}b_{22})\\
C'_y &= \texttt{partsum}(C_y,\ 2)\\
&= (a_{11}b_{12} + a_{12}b_{22},\ a_{11}b_{12} + a_{12}b_{22},\ a_{21}b_{11} + a_{22}b_{21},\ a_{21}b_{11} + a_{22}b_{21})\\
C'_x * X &= (a_{11}b_{11} + a_{12}b_{21},\quad 0,\quad 0,\quad a_{21}b_{12} + a_{22}b_{22}) = C''_x\\
C'_y * \overline{X} &= (0,\quad a_{11}b_{12} + a_{12}b_{22},\quad a_{21}b_{11} + a_{22}b_{21},\quad 0) = C''_y\\
C''_x + C''_y &= (a_{11}b_{11} + a_{12}b_{21},\ a_{11}b_{12} + a_{12}b_{22},\ a_{21}b_{11} + a_{22}b_{21},\ a_{21}b_{12} + a_{22}b_{22})\\
&= C'
\end{aligned}$$

It seems premature to go too deeply into this representation as it seems more complex and might use more memory by default because of the longer vectors.

# 34 Shannon Entropy

Suppose we have a discrete random variable $X$ with possible outcomes $x_1, x_2, \ldots, x_n$ each with corresponding probabilities $P(x_1), P(x_2), \ldots, P(x_n)$.

The **Shannon entropy**[1] is defined as

$$H(x) := -\sum_{i=1}^{n} P(x_i) \log P(x_i),$$

where $P(x_i)$ represents the probability of each outcome $x_i$.

First, commenting on the input: 1. Shannon entropy only works on a probability distribution, so all the input must be non-negative and add up to 1. 2. For sake of focus, we will assume that *this is already solved.* 3. Otherwise, this seems to be a routine case of a "normalization step" where each user input is divided by their sum, which is a combination of a **sum** and a **division** of the form

$$P(x_i) = \frac{x_i}{\sum_{j=1}^{n} x_j}.$$

We should then end up with a vector $x = (P(x_1), \ldots, P(x_n))$.

Now, we need the logarithms of the probabilities. These can also be done in various ways; when applicable, we black-box the details, deferring to previous sections: 1. we can use a Taylor series approximation, centered necessarily away from 0. 2. we can have the user simply give the $\log(x_i)$ as part of their input.

In any case, we should end up with a vector $l$ with each $l_i = \log(P(x_i))$.

Then it suffices to compute the element-wise product $x * l * (-1)$ to obtain a single vector, after which using a sum operation gives

$$H(x) := -\sum_{i=1}^{n} P(x_i) \log P(x_i)$$

(or more precisely, a vector where every element is $H(x)$).

The main difficulties while computing entropy arise in situations where $x_i = 0$. In this case, L'Hopitals rule dictates that the correct contribution of the $-P(x) \log P(x) = 0 * (\infty)$ should

[1] **Entropy (information theory)** available from https://en.wikipedia.org/wiki/Entropy_(information_theory)

be 0. We have two suggestions, and it is not clear to us which one is better (it is likely to be very implementation-dependent): 1. "iron out" the eventual answer with multiplication by an `equal(x_i, 0)` vector at the very end (before the summation), which would ensure all elements coming from an $x_i$ very close to 0 actually contributes 0; 2. interrupt the computation right before the log() step by **adding** the `equal(x_i, 0)` vector, which would switch the $x_i$s close to 0 to close to 1 (therefore making the $\log(x_i) \approx 0$.

In production, any method would require extensive testing to prevent blowup.

# 35 "Adaptive" Division

We cannot do If statements, but we can *encode* If statements with sign functions. Here's a design that might work and require some testing, but gives an idea of how we might do "adaptive" computations unadaptively:

1. For simplicity, we assume the number to invert, $x$, is positive and within the region $[1, 2^{16}]$.
2. We try to make a function that is essentially "the largest power of 2 smaller than $x$"

Suppose we have an idealized sign function, then consider the following sum, where $(x > k) = \texttt{sign}(x - k)$ is a sign indicator function (so it is 1 if the comparision is TRUE, and 0 otherwise):

$$-\frac{1}{2^{15}}(x > 2^{15}) - \frac{1}{2^{14}}(x > 2^{14}) + \cdots - \frac{1}{2}(x > 2) + 1.$$

Note:

1. if $2 > x \geq 1$, it returns 1 since none of the comparisons are true.
2. if $4 > x \geq 2$, then it returns $1 - 1/2 = 1/2$.
3. if $8 > x \geq 4$, then it returns $1 - 1/2 - 1/4 = 1/4$.
4. …the idea is that it will return $1/2^n$ if it is between $2^{n+1}$ and $2^n$.

This essentially creates a "guess" in $\log(b)$ steps (for transparency: each step is a sign call and so may be expensive), where $b$ is an upper bound of $x$. The method needs tweaking for numbers under 1 or negative numbers, which we omit.

# 36 Logarithm (Revisited)

In Section I we showed how to get logarithms via Taylor expansion. In addition to that standard approach, we explored a distinct iterative method that seems interesting, though not immediately practical, using some niceness particular to the log function.

Suppose we have

$$n \leq \log_2(x) < n + 1,$$

using the inverse relationship between the logarithm and exponentiation, this can be rewritten as:

$$2^n \leq x < 2^{n+1}.$$

If we define $m = \frac{x}{2^n}$ (so $x = m \cdot 2^n$), we can divide both sides by $2^n$ to obtain $1 \leq m < 2$. Using $m$, we can write

$$\log_2(x) = \log_2(m \cdot 2^n).$$

This effectively breaks up the log into two components: a real and a fractional part, which can be treated separately. In other words, this is noticing that if $x = m \cdot 2^n$, where $m \in [1, 2)$, then

$$\log_2(x) = \log_2(m \cdot 2^n) = n + \log_2(m).$$

As a first iteration of this strategy, we can approximate only in the range $[1, 2)$ and add back a scaling factor later. As an example, to compute $log_2(20)$, we can compute $log_2(1.25) + 4$ instead. The $n$ part can be given or "solved for" in a method similar to the previous section VII.3 to "detect" the largest $n$ such that $2^n < x$.

With the strategy just mentioned, we break the log function into an integer and fractional part. Afterwards, we can deal with the fractional part with a "fixed point iteration" method using binary representations.

Suppose $y = \log_2(m)$, and written as $y = 0.b_1b_2b_3\cdots$, (in other words, $y = b_1/2 + b_2/4 + b_3/8\cdots$). Note that if we square $m$, $y$ shifts by exactly one decimal point to

$$2y = b_1.b_2b_3b_4\cdots.$$

As a result, we are able to find the binary represenation, one bit at a time. For example, if $u = m^2$, then $u \in [1^2, 2^2) = [1, 4)$. This allows for $\log_2(u) \in [0, 2)$ This process helps us create our iteration:

$$|b_k| = \begin{cases} 0, & \text{if } u_k < 2 \to \log_2(u_{k-1}) < 1 \\ 1, & \text{if } u_k \geq 2 \to \log_2(u_{k-1}) \geq 1 \end{cases},$$

after which we can recover

$$\sum_{i=-m}^{k} b_i \, 2^i, \quad \text{where } b_i \in \{0, 1\}.$$

**Concrete Example**

We work through some explicit initial iterations as an illustrative practice. Take $x = 0.75$. We aim to obtain an approximation of $\log_2(0.75)$. Rewriting $x$ in the desired form $x = m * 2^n$, we see that $n = -1$ and $m = 1.5$.
In the first iteration, squaring $m$ gives $2.25 > 2$ hence the first bit is 1, and we must renormalize, dividing 2.25 by 2 resulting in $1.125 = u_1$. For the second iteration, squaring 1.125 we get $1.265625 < 2$ so our next bit is 0 and we do not need to renormalize.
If we continue iterating twice more we would obtain the binary digits 1001, which we interpret as the fractional part of a binary number, corresponding to the value 0.5625. This is our approximation for $\log_2(m)$. We now add $n = -1$ to give an initial approximation for $x$ as $-0.4375$. For reference, the actual value is approximately $-0.4150$. We can get closer to the actual value of $x$ with a higher number of iterations, but recall that with more iterations, we naturally have performed more computations, so error accumulates.

| $k$ | $u_{k-1}$ | $u_{k-1}^2$ | bit $b_k$ | $u_k$ |
|---|---|---|---|---|
| 1 | 1.5 | 2.25 | 1 | 1.125 |
| 2 | 1.125 | 1.265625 | 0 | 1.265625 |
| 3 | 1.265625 | 1.601807 | 0 | 1.601807 |
| 4 | 1.601807 | 2.565785 | 1 | 1.282892 |

The table lists the four iterations for $x = 0.75$, starting from $u_0 = m = 1.5$, with $u_k$ the value after any renormalization.

# A

For a reader desiring to take an "under the hood" look at CKKS, we provide an accelerated review here. We refer to the original CKKS paper: "Homomorphic Encryption for Arithmetic of Approximate Numbers"[1] as well as Inferati Inc.'s article "Introduction to the CKKS/HEAAN FHE Scheme"[2]. In particular, we may ask, what exactly is a CKKS **ciphertext** as a mathematical object?

Inferati describes the "main idea" of CKKS/HEAAN as combining a message/"**payload**" $\mu$ with the associated **noise**/error $e$, together generating the **plaintext** to be encryped, $\mu + e$. Importantly, we can't separate the two to their exact values after encrypting. The payload information ($\mu$, $e$, and payload/noise **budget** $q - (\mu + e)$ where $q$ is the "**ciphertext coefficient modulus**") is contained in a polynomial. One additional polynomial is used for decryption. For convenience, the polynomials are initially assumed to be of degree zero here. So, structurally (in highly simplified terms): **a ciphertext in CKKS is** (typically) **a pair of polynomials**.

Both polynomials' coefficients are bounded by $q$: as we compute on ciphertexts, the plaintext magnitude grows– we can perform computations until its coefficient values reach the specified bound. In other words, CKKS ciphertext arithmetic is performed mod $q$ in a polynomial ring. We choose a sufficiently large $q$ as to avoid "wraparound"/ output which may not make sense/be accurate to a client. Essentially, we don't want magnitude to grow too large relative to the modulus $q$.

Let's examine the plaintext and ciphertext spaces (which are nearly the same) with more specificity. These spaces include elements of the polynomial ring $R_q = \mathbb{Z}_q[x]/f(x)$ where $q$ is as above and $f(x)$ now is the **polynomial modulus**. Elements of $R_q$ are polynomials with integer coefficients bounded by $q$ as described, and with degrees bounded by the degree of $f(x)$. The most common choice for $f(x)$ is $x^n + 1$ with $n$, the "**ring dimension**", a power of 2. Why is this a nice choice?

This such $f(x)$ is the $2n$-th **cyclotomic polynomial** with roots being the **primitive** $2n$-th **roots of unity** (review below). The polynomial then defines a power of two **cyclotomic ring**. We ask ourselves, if CKKS works with polynomials, but we want to encrypt vectors with values $\in \mathbb{C}$, how do we connect those two representations? In highly simplified terms, by evaluating

[1] https://eprint.iacr.org/2016/421.pdf

[2] https://faculty.kfupm.edu.sa/coe/mfeleman/SEC595/References/Introduction%20to%20the%20CKKS-HEAAN%20FHE%20Scheme.pdf?

polynomials at the aforementioned roots $\zeta_1, \ldots, \zeta_n$ of $x^n+1$, we are able to convert polynomials into vectors. The CKKS authors introduce this concept as the **canonical embedding**.

**Definition:** A complex number $\zeta$ is an $n$-th root of unity if $\zeta^n = 1$. It is called a **primitive** $n$-th **root of unity** if $n$ is the smallest positive integer such that $\zeta^n = 1$.

**Definition:** The $n$-th **cyclotomic polynomial** is defined by

$$\Phi_n(x) = \prod_{\substack{1 \le k \le n \\ \gcd(k,n)=1}} \left(x - e^{2\pi i k/n}\right).$$

So, its roots are the primitive $n$-th roots of unity.

Referring to section 2.2 of the original CKKS paper: "The Cyclotomic Ring and Canonical Embedding"– first, let's note that the canonical embedding is defined prior to reducing coefficients mod $q$, also, the authors work with a real polynomial ring in their setup. Hence we consider the ring $S = \mathbb{R}[x]/f(x)$. Elements of $S$ can be represented as polynomials $a(X) = \sum_{j=0}^{N-1} a_j X^j$ with corresponding coefficient vector $(a_0, \ldots, a_{N-1}) \in \mathbb{R}^N$. The authors then define the canonical embedding as $\sigma(a) = (a(\zeta_M^j))_{j \in \mathbb{Z}_M^*} \in \mathbb{C}^N$. This $\sigma$ maps a polynomial $a(X) \in S$ to the vector of its evaluations at the $n$ distinct roots of $x^n + 1$. Also defined is the **canonical embedding norm** $\|a\|_\infty^{\text{can}} = \|\sigma(a)\|_\infty$ which has some useful properties the authors identify. The first property, submultiplicativity, appears particularly relevant: $\forall a, b \in S, \|a \cdot b\|_\infty^{\text{can}} \le \|a\|_\infty^{\text{can}} \cdot \|b\|_\infty^{\text{can}}$. This property allows us to bound the magnitude of polynomial products, helping us keep track of plaintext magnitude growth during CKKS computations.

# B References


[1] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in cryptology – ASIACRYPT 2017*, vol. 10624, T. Takagi and T. Peyrin, Eds., in Lecture notes in computer science, vol. 10624., Cham: Springer International Publishing, 2017, pp. 409–437. doi: 10.1007/978-3-319-70694-8_15.

[2] M. J. Flynn, "Some computer organizations and their effectiveness," *IEEE Transactions on Computers*, vol. C–21, no. 9, pp. 948–960, 1972, doi: 10.1109/TC.1972.5009071.

[3] T. Prantl, L. Horn, S. Engel, A. Bauer, and S. Kounev, "Quo vadis CKKS: Comparison of the realization of basic mathematical functions for the homomorphic cryptosystem CKKS using de bello and polynomial approximations," *Journal of Information Security and Applications*, vol. 93, p. 104110, 2025, doi: 10.1016/j.jisa.2025.104110.

[4] T. Prantl *et al.*, "De bello homomorphico: Investigation of the extensibility of the OpenFHE library with basic mathematical functions by means of common approaches using the example of the CKKS cryptosystem," *International Journal of Information Security*, vol. 23, no. 2, pp. 1149–1169, Apr. 2024, doi: 10.1007/s10207-023-00781-0.

[5] B. Knott, S. Venkataraman, A. Hannun, S. Sengupta, M. Ibrahim, and L. van der Maaten, "CrypTen: Secure multi-party computation meets machine learning," no. arXiv:2109.00984, 2022, doi: 10.48550/arXiv.2109.00984.

[6] K. E. Batcher, "Sorting networks and their applications," in *Proceedings of the AFIPS spring joint computer conference*, 1968, pp. 307–314. doi: 10.1145/1468075.1468121.

[7] F. H. Federico Mazzone Maarten Everts, "Efficient ranking, order statistics, and sorting under CKKS." Available: https://www.usenix.org/system/files/conference/usenixsecurity25/sec25cycle1-prepub-165-mazzone.pdf

[8] H. Chung, H. Kim, Y.-S. Kim, and Y. Lee, "Amortized large look-up table evaluation with multivariate polynomials for homomorphic encryption." Cryptology ePrint Archive, Paper 2024/274, 2024. Available: https://eprint.iacr.org/2024/274